



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Sviluppo e sperimentazione di CodeSmile: un plugin per l'analisi automatica dei Machine Learning Code smell

RELATORE

Prof. Andrea De Lucia

Dott. Gilberto Recupito

CANDIDATO

Daniele Pio Scaparra

Matricola: 0512116260

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

Abstract

L'integrazione sempre più diffusa di componenti di Machine Learning (ML) nei sistemi software tradizionali ha introdotto nuove forme di debito tecnico, note come Machine Learning-specific Code smell (ML-CSs). Pur non costituendo veri e propri bug, gli ML-CSs rappresentano pratiche di progettazione e implementazione subottimali che possono compromettere nel tempo la manutenibilità, l'affidabilità e le prestazioni dei sistemi.

In questo studio viene presentato CodeSmile, un plugin sviluppato per supportare gli sviluppatori nell'identificazione automatica degli ML-CSs direttamente all'interno dell'ambiente di sviluppo. CodeSmile offre funzionalità di analisi sia in tempo reale che su richiesta, consentendo agli utenti di rilevare tali problematiche a diversi livelli di granularità, dall'intero progetto a file singoli o insiemi di file selezionati. L'obiettivo del plugin è favorire l'integrazione di pratiche di quality assurance continua nei flussi di lavoro di sviluppo di sistemi basati su ML.

Per valutare l'efficacia e l'utilità percepita dello strumento, è stato progettato un esperimento controllato che coinvolge sviluppatori nell'ambito Machine Learning. I partecipanti sono stati chiamati a svolgere compiti di rilevamento dei code smell sia manualmente che con il supporto di CodeSmile, permettendo così un confronto diretto in termini di accuratezza, efficienza e percezione dell'esperienza d'uso. Lo user study prevede una fase di pre-screening per valutare le competenze dei partecipanti, una fase di esecuzione dei compiti alternando le modalità con e senza strumento, e una fase finale di raccolta di feedback soggettivi tramite questionario.

I risultati attesi mirano a fornire evidenze empiriche sull'impatto dell'adozione di strumenti di supporto alla rilevazione degli ML-CSs, contribuendo alla definizione di pratiche e strumenti per migliorare la qualità dei sistemi software che integrano componenti di Machine Learning.

Indice

Elenco delle Figure	iv
Elenco delle Tabelle	vi
1 Introduzione	1
1.1 Contesto Applicativo	1
1.2 Motivazioni e obiettivi	2
1.3 Risultati attesi e ottenuti	3
1.4 Struttura della tesi	4
2 Background e Stato dell'Arte	5
2.1 Code smell e ML-CSs: definizioni e tassonomie	5
2.2 Technical Debt nei sistemi ML/ AI	6
2.3 Evidenze empiriche su qualità nei progetti ML/DL/RL	8
2.4 Cataloghi di problemi e pratiche di qualità	9
2.5 Strumenti e approcci di detection	11
2.5.1 Tipo di analisi	11
2.5.2 Modalità di rilevamento	11
2.5.3 Tecnologie e strumenti	12
2.5.4 Esempi di strumenti rilevanti	12
2.6 Sintesi critica e gap	14
3 Il Plugin CodeSmile	17
3.1 Introduzione	17
3.2 Requisiti del sistema	18
3.3 Architettura e sottosistemi	20
3.4 Funzionalità principali	24

3.5	Implementazione	26
3.6	Pattern architetturali e rilascio del software	27
4	Esperimento di Valutazione	28
4.1	Introduzione	28
4.2	Design dello Studio	30
4.3	Pre-Screening Survey	33
4.4	Testing & Evaluation	35
4.5	Comparison & Feedback Survey	37
4.6	Metriche di Analisi	38
4.6.1	Metriche oggettive	38
4.6.2	Metriche soggettive	39
4.7	Sintesi del Design	39
5	Analisi dei Risultati	41
5.1	Introduzione	41
5.2	Profilo dei Partecipanti	41
5.3	Metriche Oggettive dell'Esperimento	45
5.3.1	Confronto complessivo Manuale vs Tool	45
5.3.2	Tempi di analisi	47
5.3.3	Fiducia e comportamento per categoria di file	49
5.4	Analisi per Tipologia di Smell	51
5.5	Percezione di Pericolosità e Frequenza degli smell	55
5.6	Consapevolezza ed Effetti Formativi	57
5.7	Metriche Soggettive dell'Esperimento	61
5.7.1	Preferenze di Analisi e Raccomandazione	61
5.7.2	Utilità Percepita e Accordo con le Affermazioni	61
5.7.3	Conoscenza Altre Problematiche e Motivazioni Preferenza Tool	63
5.8	Discussione Complessiva	65
6	Conclusioni e Sviluppi Futuri	68
6.1	Conclusioni	68
6.2	Sviluppi Futuri	69

Bibliografia

71

Elenco delle figure

2.1	Panorama degli strumenti: copertura vs modalità di detection.	16
3.1	Panoramica del plugin CodeSmile integrato in PyCharm.	18
3.2	UML Component Diagram dei sottosistemi di CodeSmile	22
3.3	Architettura fisica: interazione tra il plugin (PyCharm) e il backend Python.	23
4.1	Schema del flusso sperimentale: Pre-Screening → Testing & Evaluation → Comparison & Feedback	33
5.1	Profilo anagrafico dei partecipanti	43
5.2	Autovalutazione della conoscenza delle fasi della pipeline ML	43
5.3	Familiarità dichiarata con le principali librerie Python per ML	44
5.4	Livello di conoscenza dei code smell in ambito ML	44
5.5	Conoscenza di strumenti di rilevamento di code smell	44
5.6	Confronto complessivo tra analisi manuale e con tool per categoria di esito	47
5.7	Boxplot dei tempi medi di analisi manuale	48
5.8	Boxplot dei tempi medi di analisi con il tool	49
5.9	Boxplot della fiducia riposta nel tool da parte dei partecipanti	50
5.10	Confusion matrix relativa all'analisi manuale degli smell	52
5.11	Confusion matrix relativa all'analisi con il tool	54
5.12	Boxplot della pericolosità percepita dei diversi ML-CSs	56
5.13	Boxplot della frequenza percepita dei diversi ML-CSs	57
5.14	smell a cui i partecipanti dichiarano di prestare maggiore attenzione in futuro	59

5.15	Categorie di consapevolezza ed effetti formativi percepiti dai partecipanti. Il grafico mostra le diverse dimensioni in cui i partecipanti hanno dichiarato di aver beneficiato dell'esperimento	59
5.16	Problematiche percepite come più gravi per ciascun smell	60
5.17	Distribuzione delle risposte su quale smell merita maggiore/minore attenzione	60
5.18	Valutazione dell'utilità percepita del tool	62
5.19	Livello di accordo con le affermazioni sull'esperienza d'uso del tool .	63
5.20	Conoscenza di altre problematiche di qualità in ambito ML dichiarata dai partecipanti	64
5.21	Valutazioni dei partecipanti sull'esperienza con il Tool	64

Elenco delle tabelle

2.1	Mapping tra categorie di AI Technical Debt e ML-CSs correlati.	8
2.2	Tabella comparativa di strumenti e approcci di detection.	13
2.3	Gap individuati e relative opportunità di ricerca.	15
3.1	Requisiti Funzionali (FR) e Non Funzionali (NFR) del plugin CodeSmile.	19
3.2	Tracciabilità requisiti-componenti del plugin CodeSmile.	20
3.3	Sottosistemi, responsabilità e requisiti soddisfatti.	24
3.4	Modalità operative di CodeSmile, caratteristiche e requisiti soddisfatti.	25
4.1	Assegnazione dei partecipanti ai gruppi sperimentali	32
4.2	File utilizzati nella fase di <i>Testing & Evaluation</i> e categoria di riferimento	35
5.1	Metriche complessive di detection in modalità manuale e con il tool .	47
5.2	Percentuali di rilevamento corretto per categoria di file (Manuale vs Tool)	50
5.3	Metriche di precision, recall e F1-score per smell (analisi manuale) . .	53
5.4	Metriche di precision, recall e F1-score per smell (analisi con tool) . .	54

CAPITOLO 1

Introduzione

1.1 Contesto Applicativo

Negli ultimi anni, l'integrazione di componenti di *Machine Learning* (ML) nei sistemi software è diventata sempre più diffusa, portando con sé nuove sfide di qualità e manutenzione. Nei progetti software tradizionali è già noto il fenomeno dei *code smell* — pattern di codice che non generano errori immediati, ma che nel tempo compromettono qualità, manutenibilità ed estensibilità [1].

Con l'avvento delle pipeline ML, che includono fasi di preparazione dei dati, addestramento e validazione dei modelli, è emersa una categoria specifica di tali difetti: i *Machine Learning-specific Code smell* (ML-CSs) [2]. Questi code smell riguardano cattive pratiche peculiari del dominio ML, come l'uso improprio delle API delle librerie di data science o configurazioni errate dei modelli, che possono influire negativamente sulle prestazioni e sull'affidabilità dei sistemi.

A differenza dei *code smell* tradizionali, gli ML-CSs coinvolgono aspetti peculiari legati alla natura *data-driven* di tali sistemi, come la gestione del flusso dei dati, la configurazione dei modelli e la correttezza del processo di addestramento. Individuarli e correggerli tempestivamente consente di ridurre l'accumulo di *debito tecnico* [3] che, se trascurato, può ostacolare la manutenzione e l'evoluzione futura del software.

Nonostante il crescente interesse verso la qualità del software ML, le pratiche e gli strumenti attualmente disponibili, oltre ad essere pochi, mostrano alcune lacune quali:

- molti metodi di rilevamento sono progettati per *code smell* generici e non considerano le peculiarità delle pipeline ML [4];
- le analisi sono spesso svolte *offline* e non integrate nell'ambiente di sviluppo, limitando il feedback immediato per professionisti dell'ambito ML;
- le funzionalità di rilevamento in tempo reale sono ancora rare, nonostante siano cruciali per prevenire l'introduzione di smell già durante la scrittura del codice.

Affrontare questi limiti significa contribuire alla diffusione di pratiche di *continuous quality assurance* anche nello sviluppo di sistemi ML, aumentando produttività, affidabilità e sostenibilità del software.

1.2 Motivazioni e obiettivi

Per rispondere a queste esigenze, questa tesi ha come obiettivo principale progettare, sviluppare e validare un plugin per *JetBrains PyCharm*¹ che estenda le funzionalità di CODESMILE, uno strumento di analisi statica per ML-CSs sviluppato e descritto da Recupito et al. nel loro studio sul ciclo di vita dei code smell specifici del Machine Learning nei sistemi abilitati al Machine Learning [5].

Le sue funzionalità chiave includono:

- Analisi dell'intero progetto, di singoli file o di insiemi di file selezionati;
- Rilevamento in tempo reale durante la scrittura del codice;
- Presentazione chiara dei risultati con suggerimenti di correzione direttamente nell'IDE.

¹*JetBrains PyCharm* è un ambiente di sviluppo integrato (IDE) focalizzato principalmente sul linguaggio Python e largamente utilizzato nella comunità di machine learning e data science grazie al supporto nativo per librerie scientifiche, strumenti di debugging avanzati e gestione dei virtual environment.

Sulla base di questi obiettivi, il presente lavoro si propone di rispondere al seguente interrogativo:

Un plugin integrato nell'IDE, in grado di rilevare ML-Specific Code smell sia on-demand che in tempo reale, può supportare gli sviluppatori nella gestione di cattive pratiche e contribuire a migliorare la qualità dei sistemi di Machine Learning?

1.3 Risultati attesi e ottenuti

Il lavoro svolto ha portato alla realizzazione di un plugin funzionante che rende disponibile l'analisi di ML-CSs direttamente in PyCharm, sia in modalità *on-demand* sia *real-time*.

Per valutarne efficacia e utilità percepita, è stato progettato un esperimento controllato articolato in tre fasi:

1. **Pre-screening survey** per raccogliere informazioni sul background e le competenze dei partecipanti;
2. **Sessioni di rilevamento** svolte manualmente e con il plugin, in ordine bilanciato tra i gruppi;
3. **Questionario comparativo** per misurare accuratezza, tempi, usabilità e utilità percepita.

I risultati attesi includono:

- aumento dell'accuratezza nel rilevamento di ML-CSs;
- riduzione dei tempi di individuazione;
- miglioramento della consapevolezza degli sviluppatori riguardo le cattive pratiche nelle pipeline ML.

1.4 Struttura della tesi

La tesi è strutturata come segue:

- **Capitolo 2 – Background e Stato dell’Arte:**
Descrive concetti di base sui code smell e ML-CSs, legame con il debito tecnico, evidenze empiriche e strumenti di rilevamento;
- **Capitolo 3 – Il Plugin CodeSmile:**
Descrive in maniera accurata requisiti, architettura e implementazione del plugin sviluppato;
- **Capitolo 4 – Esperimento di Valutazione:**
Descrive dettagliatamente la metodologia utilizzata per la stesura dello user study, fornendo dettagli anche sul metodo di somministrazione dello stesso e delle metriche utilizzate ;
- **Capitolo 5 – Analisi dei Risultati:**
Fornisce una presentazione e discussione dei risultati ottenuti dopo la somministrazione dell’esperimento;
- **Capitolo 6 – Conclusioni e Sviluppi Futuri:**
Vengono esposte sintesi dei contributi e prospettive di estensione del lavoro.

Background e Stato dell'Arte

2.1 Code smell e ML-CSs: definizioni e tassonomie

Il termine *code smell* è stato introdotto da Fowler [1] per indicare porzioni di codice che, pur non causando errori immediati, rappresentano segnali di potenziali problemi di progettazione o implementazione. Questi pattern, se trascurati, possono compromettere la qualità del software e aumentarne i costi di manutenzione nel tempo. I *code smell* non sono bug in senso stretto, ma “campanelli d’allarme” che richiedono attenzione per evitare l’accumulo di debito tecnico.

Con la crescente integrazione di componenti di *Machine Learning* (ML) nei sistemi software, la ricerca ne ha individuato una sottocategoria specifica appartenente a questo dominio: i *Machine Learning-specific Code smell* (ML-CSs). Zhang et al. [2] definiscono un ML-CS come una cattiva pratica di sviluppo che emerge in una fase della pipeline ML — dalla preparazione dei dati, all’addestramento, alla validazione e inferenza — e che può compromettere la manutenibilità, l’affidabilità o le prestazioni del sistema. Il catalogo proposto da Zhang identifica 22 tipologie di ML-CSs, raggruppate in macro-categorie che includono:

- problemi nella gestione dei dati (ad esempio uso scorretto delle API per la

manipolazione di dataset);

- cattive pratiche nell'addestramento (come gestione inadeguata dei gradienti);
- configurazioni non esplicite o incoerenti;
- fragilità nella gestione di modelli e pipeline.

Esempi riportati in letteratura includono operazioni inefficaci o ridondanti sui dati, mancata inizializzazione o pulizia di variabili cruciali per l'addestramento, e utilizzo di metodi in modo non conforme alle raccomandazioni delle librerie ML [2, 5]. Questi smell, sebbene non provochino un malfunzionamento immediato, possono rendere il sistema meno robusto e più difficile da mantenere.

Diversi studi empirici hanno confermato la presenza diffusa degli ML-CSs. Recupito et al. [5] hanno analizzato progetti open source identificando pattern ricorrenti e valutandone la frequenza e il ciclo di vita; Jebnoun et al. [6] hanno osservato la coesistenza di smell "classici" e ML-specifici nel deep learning; Cardozo et al. [7] hanno riscontrato code smell tipici anche in contesti di reinforcement learning. Queste evidenze dimostrano che il problema non è circoscritto a un sottoinsieme ristretto di progetti, ma è trasversale a diverse aree del Machine Learning.

2.2 Technical Debt nei sistemi ML/AI

Avgeriou et al. [8] definiscono il *Technical Debt* (TD) come «qualsiasi costrutto di progettazione o implementazione che risulti conveniente nel breve termine, ma che crei un contesto tecnico tale da rendere un cambiamento futuro più costoso o addirittura impossibile». In altri termini, il TD rappresenta scelte progettuali o implementative che, pur offrendo benefici immediati, introducono vincoli o complessità che aumentano il costo e la difficoltà di modifiche future. Recenti indagini, come quella di Côté et al. [9], confermano la diffusione di problematiche di qualità nei sistemi ML, evidenziando come queste possano tradursi in diverse forme di debito tecnico.

Nel contesto del *Machine Learning*, il TD assume forme peculiari, spesso meno visibili rispetto allo sviluppo software tradizionale. Sculley et al. [3] hanno coniato il concetto di *Hidden Technical Debt* nei sistemi ML, individuando categorie quali:

- **Data Debt:** problemi nella gestione, qualità e tracciabilità dei dati;
- **Configuration Debt:** configurazioni e iperparametri gestiti in modo manuale o non documentato;
- **Glue Code e Pipeline Jungles:** script e codice di collegamento non strutturati;
- **Entanglement:** interdipendenze non intenzionali tra componenti della pipeline.

Bogner et al. [10] hanno ampliato questa visione proponendo una tassonomia di debiti tecnici specifici per l'AI, includendo non solo debiti sui dati, sulla configurazione e sul codice, ma anche sul modello e di natura etica. Questa categorizzazione evidenzia come molti *Machine Learning-specific Code smell* (ML-CSs) possano essere considerati indicatori precoci di TD: ad esempio, un *Chain Indexing* o un *Columns and Datatype Not Explicitly Set* possono segnalare **Data Debt**, mentre un *Gradient Not Cleared* può indicare **Configuration Debt**.

Per rendere evidente il legame tra teoria e pratica, la Tabella 2.1 mostra una mappatura esemplificativa tra categorie di AI Technical Debt e ML-CSs tipicamente associati.

Tabella 2.1: Mapping tra categorie di AI Technical Debt e ML-CSs correlati.

Categoria TD	Descrizione	Esempi di ML-CSs correlati
Data Debt	Problemi nella gestione, qualità e tracciabilità dei dati	Chain Indexing; Columns and Datatype Not Explicitly Set
Configuration Debt	Gestione inadeguata di configurazioni e iperparametri	Gradients Not Cleared Before Backward Propagation; PyTorch Call Method Misused
Code Debt	Scarsa qualità o manutenibilità del codice	In-place APIs Misused; TensorArray Not Used
Model Debt	Modelli mal documentati o non aggiornati	Obsolete Model Without Retraining; Hard-coded Parameters
Ethics Debt	Questioni etiche e di bias nei dati/modelli	Unbalanced Data Without Mitigation; Lack of Model Auditing

Nota: La mappatura è esemplificativa e mostra possibili corrispondenze tra categorie di debito tecnico e ML-CSs rilevabili.

Questo collegamento concettuale permette di interpretare gli ML-CSs non solo come problemi puntuali di codice, ma come segnali misurabili di un potenziale accumulo di debito tecnico più ampio. La rilevazione automatica di tali smell può quindi fungere da strategia preventiva per ridurre il TD nei sistemi ML, migliorandone la sostenibilità nel lungo periodo.

2.3 Evidenze empiriche su qualità nei progetti ML/DL/RL

La letteratura recente mostra un crescente interesse verso l'analisi empirica della qualità nei progetti di *Machine Learning* (ML), *Deep Learning* (DL) e *Reinforcement Learning* (RL). Tali studi si concentrano sia sull'identificazione e classificazione dei *Machine Learning-specific Code smell* (ML-CSs), sia sulla valutazione del loro impatto sulla manutenibilità, sull'affidabilità e sulle prestazioni dei sistemi.

Recupito et al. [5] hanno condotto un'analisi su un ampio insieme di repository open source, individuando la presenza diffusa di ML-CSs e valutandone il ciclo di

vita e la severità percepita. I risultati mostrano che alcuni smell, come *Chain Indexing* o *Gradients Not Cleared Before Backward Propagation*, tendono a persistere più a lungo nel codice rispetto ad altri, indicando una difficoltà intrinseca nella loro individuazione e correzione.

Jebnoun et al. [6] hanno analizzato progetti DL, riscontrando come smell tipici dello sviluppo tradizionale, ad esempio il *Long Method* o il *God Class*, coesistano con smell specifici del dominio ML. Questa coesistenza evidenzia la necessità di strumenti in grado di rilevare entrambe le tipologie, garantendo un controllo della qualità più completo.

Cardozo et al. [7] hanno approfondito il caso del RL, mostrando che anche in questo ambito emergono code smell peculiari, spesso legati alla gestione dello stato e alla configurazione degli agenti. Questi risultati suggeriscono che le problematiche di qualità non siano limitate a un singolo paradigma ML, ma siano trasversali a diverse aree applicative.

Infine, Côté et al. [9] hanno condotto una survey su sviluppatori e ricercatori, rilevando che i problemi di qualità percepiti nei sistemi ML sono frequentemente connessi a debiti tecnici sui dati, sulla configurazione e sui modelli. La survey evidenzia come tali problematiche impattino negativamente sia sul processo di sviluppo che sulla capacità di mantenere e riutilizzare i modelli nel tempo.

Queste evidenze empiriche rafforzano l'idea che la gestione proattiva degli ML-CSs sia un fattore chiave per il mantenimento della qualità del software ML e per la riduzione del debito tecnico nei progetti reali.

2.4 Cataloghi di problemi e pratiche di qualità

Negli ultimi anni, diversi studi hanno proposto cataloghi strutturati di *Machine Learning-specific Code smell* (ML-CSs) e di pratiche di sviluppo mirate al miglioramento della qualità nei sistemi ML. Tali cataloghi hanno lo scopo di fornire a sviluppatori e ricercatori una classificazione sistematica delle cattive pratiche più comuni e delle strategie per prevenirle o correggerle.

Il lavoro di Zhang et al. [2] rappresenta uno dei contributi fondamentali in questo ambito, presentando un catalogo di 22 ML-CSs suddivisi in quattro macro-categorie:

- **Data-related smell:** cattive pratiche nella gestione e manipolazione dei dati (es. uso improprio di API di librerie come *Pandas* o *NumPy*);
- **Model-related smell:** problematiche nella definizione, addestramento e validazione dei modelli (es. parametri non inizializzati, configurazioni incoerenti);
- **Training-related smell:** errori o inefficienze durante la fase di addestramento (es. mancata pulizia dei gradienti, batch size inappropriato);
- **Evaluation-related smell:** cattive pratiche nella valutazione o nell'uso dei modelli (es. uso di metriche non rappresentative, valutazioni su dataset non coerenti).

Recupito et al. [5] hanno esteso questa analisi studiando il ciclo di vita degli ML-CSs nei repository open source, fornendo dati sulla frequenza e persistenza di ciascuna tipologia. I risultati suggeriscono che alcune categorie di smell, in particolare quelli legati alla gestione dei dati e alla configurazione, tendono a essere più persistenti e difficili da eliminare.

Oltre all'individuazione dei problemi, la letteratura ha proposto anche pratiche di qualità per mitigarne l'impatto. Ad esempio, Côté et al. [9] raccomandano l'adozione di strumenti di analisi automatica integrati nei flussi di sviluppo e il miglioramento della documentazione relativa a dati e modelli. Queste pratiche, se adottate in maniera sistematica, possono ridurre significativamente il debito tecnico e migliorare la manutenibilità del software ML.

Infine, alcuni cataloghi includono raccomandazioni per l'educazione e la formazione degli sviluppatori, sottolineando come la consapevolezza dei ML-CSs e delle relative pratiche di mitigazione sia cruciale per il mantenimento della qualità a lungo termine. L'adozione di tali cataloghi non solo favorisce la standardizzazione delle pratiche di qualità, ma può anche guidare lo sviluppo di strumenti di rilevamento e correzione automatica.

2.5 Strumenti e approcci di detection

La rilevazione dei *Machine Learning-specific Code smell* (ML-CSs) rappresenta una sfida sia tecnica che metodologica. In letteratura, gli approcci proposti possono essere classificati in base a tre dimensioni principali.

2.5.1 Tipo di analisi

- **Analisi statica:** ispezione del codice senza eseguirlo, spesso basata su regole predefinite (*rule-based*) che identificano pattern sintattici o strutturali riconducibili a ML-CSs [2, 5].
- **Analisi dinamica:** osservazione del comportamento del programma in esecuzione, utile per individuare smell legati a configurazioni, flussi di dati o stato del modello durante il training.
- **Approcci ibridi:** combinazione di analisi statica e dinamica per ottenere una copertura più ampia e rilevare smell sia strutturali che di comportamento.

2.5.2 Modalità di rilevamento

- **Offline detection:** esecuzione dell'analisi come passaggio separato dal ciclo di sviluppo, tipicamente tramite script o pipeline CI/CD. È la modalità prevalente negli strumenti esistenti [2, 5].
- **Real-time detection:** integrazione diretta nell'IDE, fornendo feedback immediato mentre il codice viene scritto. Questa modalità, seppur cruciale per la prevenzione, è ancora poco diffusa.
- **On-demand detection:** esecuzione manuale dell'analisi su richiesta dello sviluppatore, ad esempio tramite comandi o menu contestuali in un IDE.

2.5.3 Tecnologie e strumenti

- **Approcci rule-based:** derivano regole dai cataloghi di ML-CSs (ad esempio [2]) e le implementano in strumenti di analisi statica (AST parsing, pattern matching, linting avanzato).
- **Approcci machine learning-based:** addestrano modelli di classificazione (Decision Tree, SVM, Random Forest, Naive Bayes, XGBoost) su dataset annotati, spesso utilizzando tecniche di bilanciamento come SMOTE per migliorare le prestazioni sui dataset sbilanciati [11].
- **Approcci deep learning-based:** sfruttano architetture come Bi-LSTM, GRU o CNN per catturare sia la sequenza di token che la struttura sintattica del codice, raggiungendo accuratezze elevate (fino al 98–100%) su dataset bilanciati.
- **Approcci ensemble:** come ENSESMELL [12], che combinano tecniche di *ensemble learning* e modelli pre-addestrati per il codice (code embeddings), ottenendo miglioramenti di accuratezza fino al 28% rispetto a metodi puramente strutturali.

2.5.4 Esempi di strumenti rilevanti

- **DSLint** [4]: strumento di analisi statica *rule-based* sviluppato per rilevare ML-CSs tramite regole definite a partire da un catalogo specifico.
- **CODESMILE** [5]: strumento di analisi statica *offline* sviluppato per rilevare ML-CSs in progetti Python e produrre statistiche aggregate, senza integrazione IDE o rilevamento in tempo reale.
- **MLpylint** [13]: tool di analisi statica *rule-based* progettato per identificare ML-CSs in progetti Python, valutato su 160 repository e validato tramite esperti.
- **MLScent** [14]: tool basato su AST con 76 rilevatori di anti-pattern per PyTorch, TensorFlow, Scikit-learn e altre librerie di data science, validato con studi quantitativi e qualitativi.

- **ML Code Smell Detector** [13]: pacchetto Python con detector per ML-CSs specifici di framework come Pandas, NumPy e Hugging Face.
- **Pysmell**: strumento generico per il rilevamento di code smell in Python, utile come termine di confronto con i tool specifici per ML.

Questi esempi mostrano un panorama eterogeneo di approcci: *rule-based*, *ML-based*, *DL-based* e ibridi, ciascuno con vantaggi e limiti. Tuttavia, emerge una tendenza comune: la netta prevalenza della modalità **offline**, con una carenza di strumenti che supportino il **real-time detection** integrato negli ambienti di sviluppo. Questa lacuna è particolarmente rilevante in ottica di prevenzione, poiché il rilevamento immediato può evitare l'introduzione di smell già durante la scrittura del codice.

La Tabella 2.2 sintetizza i principali strumenti e approcci di detection emersi in letteratura, classificandoli in base al tipo di analisi, alla modalità di esecuzione e alle tecnologie adottate. Tale sintesi consente di evidenziare a colpo d'occhio le aree già coperte e quelle che rappresentano opportunità di ricerca e sviluppo.

Tabella 2.2: Tabella comparativa di strumenti e approcci di detection.

Strumento	Tipo di analisi	Modalità	Tecnologia
Dslinter	Statica (rule-based)	Offline	AST parsing, regole da catalogo ML-CSs
CODESMILE	Statica (rule-based)	Offline	AST parsing, regole da catalogo ML-CSs
MLpylint	Statica (rule-based)	Offline	Regole specifiche per ML in Python
MLScent	Statica (AST-based)	Offline	76 rilevatori per ML frameworks

ML Detector	Code Smell	Statica (framework-specific)	Offline	Detector per Pandas, NumPy, Hugging Face
Pysmell		Statica (generico)	Offline	Generic Python smell detector
Ensesmell		Statica + ML (ensemble)	Offline	Ensemble learning + code embeddings
Modelli ML classici		ML supervisionato	Offline On-demand	Decision Tree, SVM, Random Forest, XGBoost
Modelli DL (LSTM, CNN)	(Bi-GRU,	Deep Learning	Offline On-demand	Architetture seq./conv.

***Nota:** La tabella riassume strumenti e approcci rilevanti, con classificazione per tipo di analisi, modalità di esecuzione e tecnologie impiegate.*

2.6 Sintesi critica e gap

L'analisi dello stato dell'arte mostra come la ricerca sui *Machine Learning-specific Code smell* (ML-CSs) abbia compiuto significativi progressi negli ultimi anni, sia nella definizione concettuale, sia nello sviluppo di strumenti di rilevamento. Possiamo sintetizzare i principali contributi su tre direttrici:

1. **Definizione e classificazione** — Zhang [2] e Recupito et al. [5] hanno proposto cataloghi strutturati di ML-CSs, fornendo tassonomie chiare e descrizioni delle principali tipologie. Questi lavori hanno standardizzato la terminologia e posto le basi per la creazione di tool di detection.
2. **Evidenze empiriche** — Studi come Jebnoun et al. [6], Cardozo et al. [7] e Côté et al. [9] hanno confermato l'elevata diffusione degli ML-CSs in diversi

domini (ML, DL, RL) e il loro impatto negativo sulla qualità e manutenibilità dei sistemi.

3. **Strumenti di detection** — Numerosi strumenti sono stati sviluppati, dai rule-based (DSLint [4], CodeSmile [5], MLpylint, MLScint [14]) agli approcci ML/DL-based (Ensesmell [12]). Tuttavia, come evidenziato nella Tabella 2.2, la quasi totalità di essi opera in modalità **offline**.

Nonostante questi progressi, emergono diverse lacune significative:

- **Carenza di real-time detection** — Come sottolineato da Côté et al. [9], mancano strumenti che forniscano feedback immediato durante la scrittura del codice.
- **Limitata integrazione nei flussi di sviluppo** — Pochi strumenti si integrano senza attrito nei workflow degli sviluppatori, ad esempio come plugin per IDE.
- **Assenza di validazioni in contesti industriali** — Bogner et al. [10] evidenziano la scarsità di valutazioni in ambienti produttivi reali.
- **Scarso uso di approcci ibridi** — Pochi esempi, come Ensesmell [12], combinano regole esplicite e modelli ML/DL, e nessuno è focalizzato su ML-CSs.

La Tabella 2.3 riassume i principali gap emersi e le opportunità di ricerca e sviluppo.

Tabella 2.3: Gap individuati e relative opportunità di ricerca.

Gap	Opportunità
Mancanza di real-time detection	Sviluppo di plugin IDE con feedback immediato
Validazioni limitate a OSS	Test e validazioni in contesti industriali reali
Scarso uso approcci ibridi	Combinazione di regole e ML/DL per alta precisione

La Figura 2.1 mostra visivamente la distribuzione degli strumenti rispetto a copertura del catalogo e modalità di detection, evidenziando come la maggior parte si concentri

nella parte bassa della matrice (offline), mentre la proposta di questa tesi (plugin PyCharm) si colloca in alto a destra, unendo ampia copertura e real-time detection.

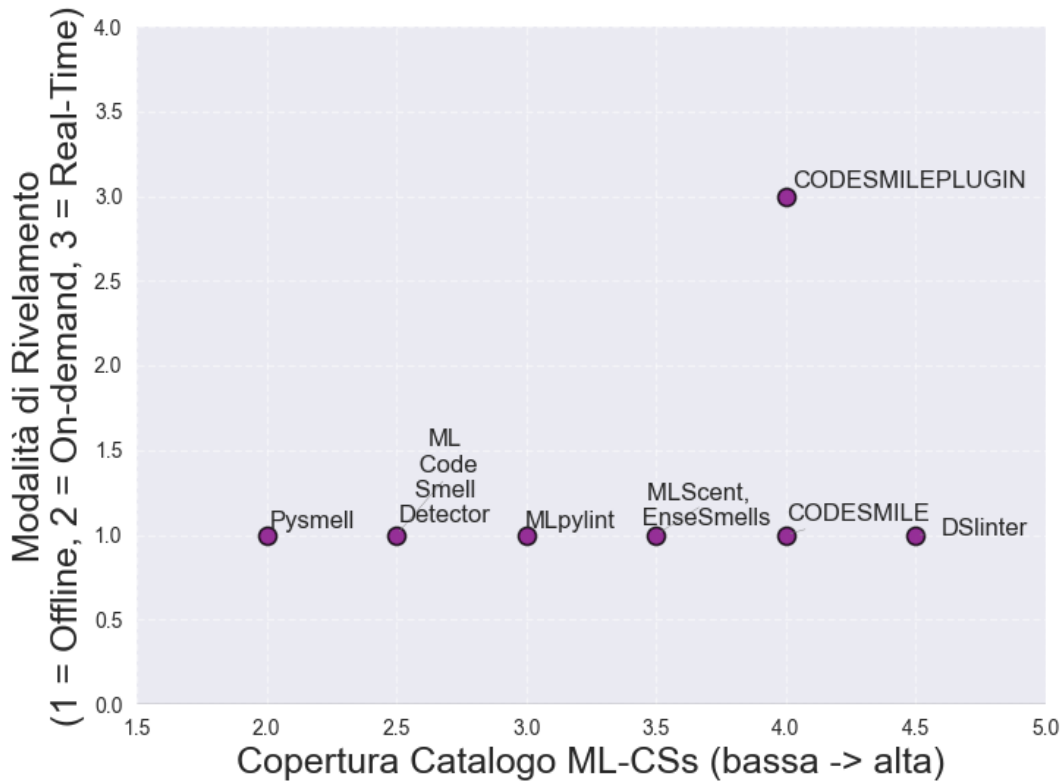


Figura 2.1: Panorama degli strumenti: copertura vs modalità di detection.

L'obiettivo di questa tesi si inserisce in questo spazio di miglioramento, proponendo lo sviluppo e la validazione di un plugin per PyCharm che estenda CodeSmile con funzionalità di rilevamento on-demand e real-time, fornendo suggerimenti di correzione direttamente nell'IDE.

Il Plugin CodeSmile

3.1 Introduzione

L'analisi dello stato dell'arte ha evidenziato importanti lacune negli strumenti di rilevamento dei *Machine Learning-specific Code smell* (ML-CSs), tra cui:

- assenza di funzionalità di rilevamento **in tempo reale** integrate negli ambienti di sviluppo (*IDE*);
- limitata integrazione nei flussi di lavoro degli sviluppatori.

Queste criticità sono state riportate da diversi studi recenti [2, 5, 9], e sono coerenti con quanto emerso dalla revisione sistematica della letteratura condotta nel Capitolo 2.

Per affrontare tali gap, in questo lavoro di tesi è stato progettato e sviluppato **CodeSmile**, un plugin per *JetBrains PyCharm* che estende lo strumento di analisi statistica *CODESMILE* [5]. Il plugin consente l'individuazione di ML-CSs sia **on-demand** che **in tempo reale**, direttamente durante lo sviluppo del software, fornendo:

- *feedback* immediato e mirato;
- granularità di analisi (intero progetto, singolo file, set di file selezionati);

- suggerimenti di correzione direttamente nell'IDE.

L'approccio metodologico adottato si ispira ai principi della *Object-Oriented Software Engineering* di Bruegge e Dutoit [15] e della *Software Engineering* di Sommerville [16], con particolare attenzione alla modellazione UML, alla definizione dei requisiti funzionali e non funzionali, e all'uso di *design patterns* consolidati [17] per garantire estendibilità e manutenibilità del sistema.

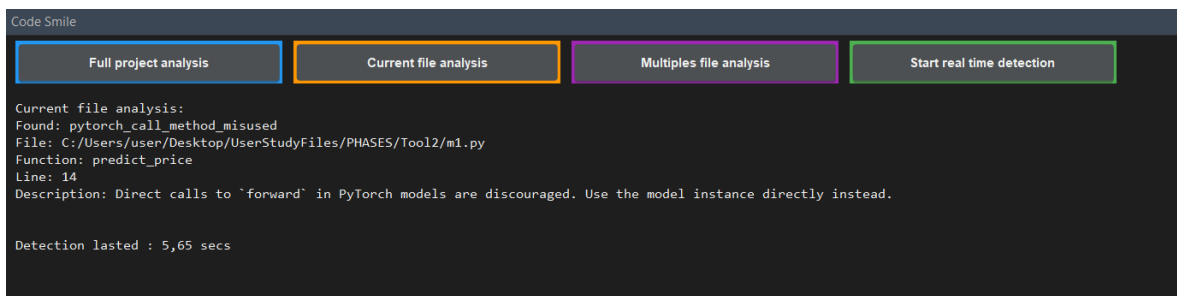


Figura 3.1: Panoramica del plugin CodeSmile integrato in PyCharm.

3.2 Requisiti del sistema

La definizione dei requisiti costituisce un momento fondamentale nel processo di sviluppo di qualsiasi sistema software, in quanto permette di fissare in maniera chiara e condivisa le funzionalità che il sistema dovrà offrire e le qualità che dovrà garantire. Seguendo le linee guida proposte da Sommerville [16] e Bruegge & Dutoit [15], si è proceduto a distinguere tra **requisiti funzionali** (*Functional Requirements*, FR) e **requisiti non funzionali** (*Non-Functional Requirements*, NFR).

I requisiti funzionali descrivono *cosa* il sistema deve fare, cioè le capacità specifiche necessarie per soddisfare gli obiettivi progettuali. Nel caso di CodeSmile, derivano direttamente dagli obiettivi identificati nell'analisi dei gap della letteratura [9, 5], che evidenziano la necessità di un rilevamento degli ML-CSs sia su richiesta che in tempo reale, con un'interfaccia integrata nell'IDE e un feedback chiaro per lo sviluppatore. I principali FR includono:

- la possibilità di analizzare progetti Python completi o singoli file e individuare automaticamente ML-CSs;

- la disponibilità di più modalità di analisi (intero progetto, file corrente, selezione multipla, real-time);
- il supporto alla rilevazione in tempo reale per prevenire l'introduzione di *smell* già in fase di scrittura del codice;
- la presentazione di messaggi chiari e localizzati direttamente nell'editor.

I requisiti non funzionali, invece, definiscono *come* il sistema deve operare, ponendo vincoli e obiettivi in termini di prestazioni, scalabilità, compatibilità e usabilità. Nel caso di CodeSmile, particolare attenzione è stata posta a garantire:

- prestazioni adeguate, evitando rallentamenti dell'IDE anche in presenza di analisi complesse;
- scalabilità, per poter integrare in futuro un motore di analisi basato su tecniche di Machine Learning e Deep Learning;
- compatibilità multiplatforma (Windows, Linux, macOS) e con le versioni più diffuse di PyCharm.

La Tabella 3.1 sintetizza i requisiti funzionali e non funzionali del plugin.

Tabella 3.1: Requisiti Funzionali (FR) e Non Funzionali (NFR) del plugin CodeSmile.

Requisito	Descrizione
FR1	Analisi di progetti Python e rilevamento di ML-CSs utilizzando il motore CODESMILE.
FR2	Interfaccia per la selezione della modalità di analisi: intero progetto, file corrente, set di file, real-time.
FR3	Supporto alla <i>real-time detection</i> per prevenire l'introduzione di smell durante la scrittura del codice.
FR4	Fornitura di <i>feedback</i> visivo nell'editor con dettagli su file, linee e tipo di smell.
NFR1	Prestazioni adeguate senza rallentare PyCharm.

NFR2	Scalabilità per future integrazioni con motori AI di analisi.
NFR3	Compatibilità multiplatforma (Windows, Linux, macOS).

Per garantire la copertura implementativa dei requisiti e facilitare le attività di verifica e validazione, si è inoltre predisposta una **matrice di tracciabilità** che mette in relazione ogni requisito con i componenti architetturali responsabili della sua implementazione. Tale approccio, suggerito anche in letteratura [16], consente di mantenere un legame diretto tra le specifiche e le scelte di design, facilitando la manutenzione futura del sistema. La Tabella 3.2 riporta questa corrispondenza.

Tabella 3.2: Tracciabilità requisiti–componenti del plugin CodeSmile.

Requisito	Componenti Implementativi
FR1 — Analisi progetti Python e detection ML-CSs	Gestione Server Python; Comunicazione di Rete
FR2 — Selezione modalità di analisi	Interfaccia Utente (Tool Window Menu)
FR3 — Real-time detection	Monitoraggio Real-Time; Comunicazione di Rete
FR4 — Feedback visivo nell’editor	Interfaccia Utente; Formattazione Risultati
NFR1 — Prestazioni	Ottimizzazione Comunicazione di Rete; Backend Python
NFR2 — Scalabilità AI	Architettura modulare; Gestione Server Python
NFR3 — Compatibilità multiplatforma	Uso API PyCharm; IntelliJ SDK

3.3 Architettura e sottosistemi

L’architettura di CodeSmile è progettata secondo un approccio modulare per favorire la manutenibilità, l’estensibilità e la chiarezza delle responsabilità, in linea

con i principi di bassa accoppiatura e alta coesione illustrati da Sommerville [16], Bruegge & Dutoit [15] e Pressman & Maxim [18]. Questa separazione permette di gestire la complessità del sistema e facilita l'integrazione di estensioni future non invasive, come un motore AI (FR2, NFR2).

Il sistema è suddiviso in cinque sottosistemi principali, ciascuno con responsabilità chiare e delimitate:

- **Interfaccia Utente – Tool Window Menu:** gestisce la selezione delle modalità di analisi, attivazione/disattivazione della *real-time detection* e visualizzazione dei risultati. Copre FR2 e FR4.
- **Gestione Server Python:** avvia, monitora e garantisce l'accessibilità del backend Flask¹ che esegue l'analisi con CODESMILE. Riferito a FR1 e NFR1.
- **Comunicazione di Rete – NetworkUtil:** invia richieste HTTP al backend e riceve i risultati, realizzando un *Facade Pattern* secondo Gamma et al. [17].
- **Formattazione dei Risultati – ReportUtil:** legge i file CSV² in output, formatta i risultati e pulisce i file temporanei. Supporta FR4.
- **Monitoraggio Real-Time – Listener:** sfrutta un *Observer Pattern* per rilevare modifiche ai file .py e attivare analisi automatiche (FR3).

Questa composizione architetture riflette pratiche consolidate per lo sviluppo di plugin integrati negli IDE, secondo linee guida JetBrains³. Il plugin è sviluppato in Kotlin⁴, scelto per la sua piena compatibilità con la piattaforma IntelliJ e le API di PyCharm.

La Figura 3.2 fornisce una rappresentazione ad alto livello dei sottosistemi di CodeSmile e delle loro relazioni. Nel diagramma UML dei componenti, ciascun

¹Flask è un micro-framework web per Python che consente di sviluppare applicazioni web e API REST in modo semplice e leggero.

²Comma-Separated Values: formato di file testuale utilizzato per memorizzare dati tabellari, in cui i valori sono separati da virgole o altri delimitatori.

³<https://plugins.jetbrains.com/docs/intellij/documentation.html>

⁴Kotlin è un linguaggio di programmazione moderno, interoperabile con Java, sviluppato da JetBrains e utilizzato per applicazioni server-side, mobile e plugin per IDE.

modulo è rappresentato come un'entità indipendente, con interfacce ben definite che regolano lo scambio di informazioni. Le connessioni tra i componenti riflettono i flussi di dati e le dipendenze logiche illustrate nella descrizione precedente: l'interfaccia utente comunica con i moduli di gestione e comunicazione, i quali interagiscono con il backend Python per l'esecuzione dell'analisi. Questa vista strutturale aiuta a comprendere come le diverse parti collaborano per realizzare le funzionalità del sistema.

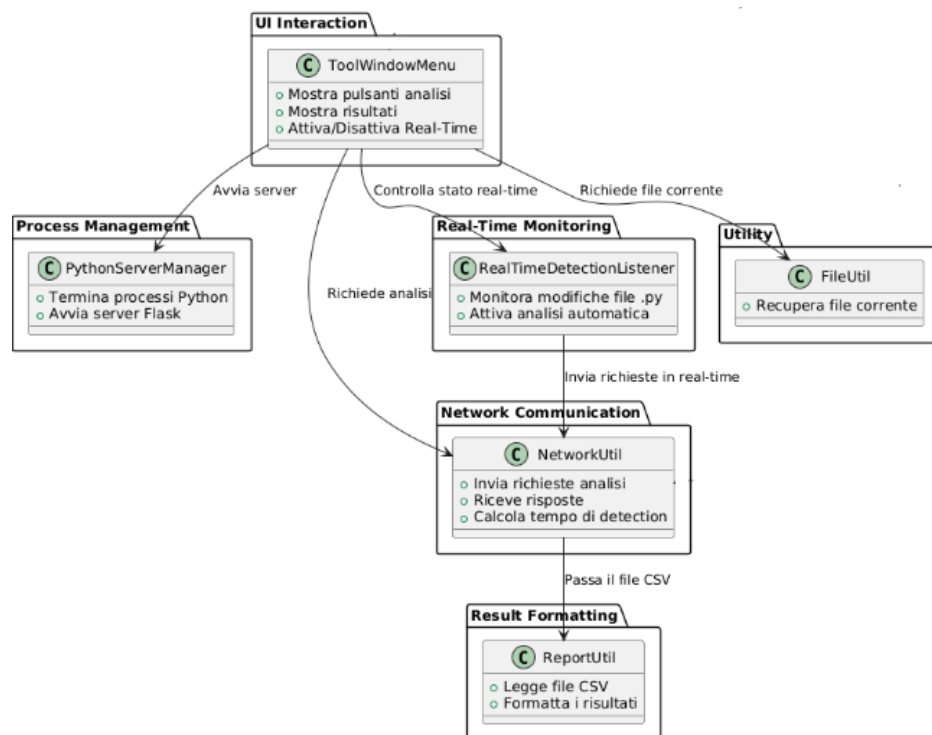


Figura 3.2: UML Component Diagram dei sottosistemi di CodeSmile .

La Figura 3.3 illustra l'architettura fisica di CodeSmile, evidenziando la disposizione dei principali componenti e le modalità di interazione tra essi. Il sistema adotta un'architettura a due livelli: **frontend**, rappresentato dal plugin integrato in PyCharm, e **backend**, costituito da un server Python Flask che esegue il motore di analisi CODESMILE. Il plugin fornisce l'interfaccia utente e le funzionalità di gestione dell'interazione, inviando richieste HTTP al backend locale. Quest'ultimo, eseguendo l'analisi dei file Python presenti nel workspace, produce un output strutturato (CSV/JSON) che viene restituito al plugin per la formattazione e la visualizzazione dei risultati.

Le frecce nel diagramma indicano i principali flussi di comunicazione: dall'utente al plugin tramite l'IDE, dal plugin al server mediante chiamate REST locali e dal backend al file system per la lettura del codice sorgente e la scrittura dei risultati. Questa organizzazione modulare, in linea con le raccomandazioni di Bruegge [15] e Pressman [18], favorisce la separazione delle responsabilità e la scalabilità del sistema, consentendo future estensioni, come l'integrazione di un motore AI per la detection avanzata.

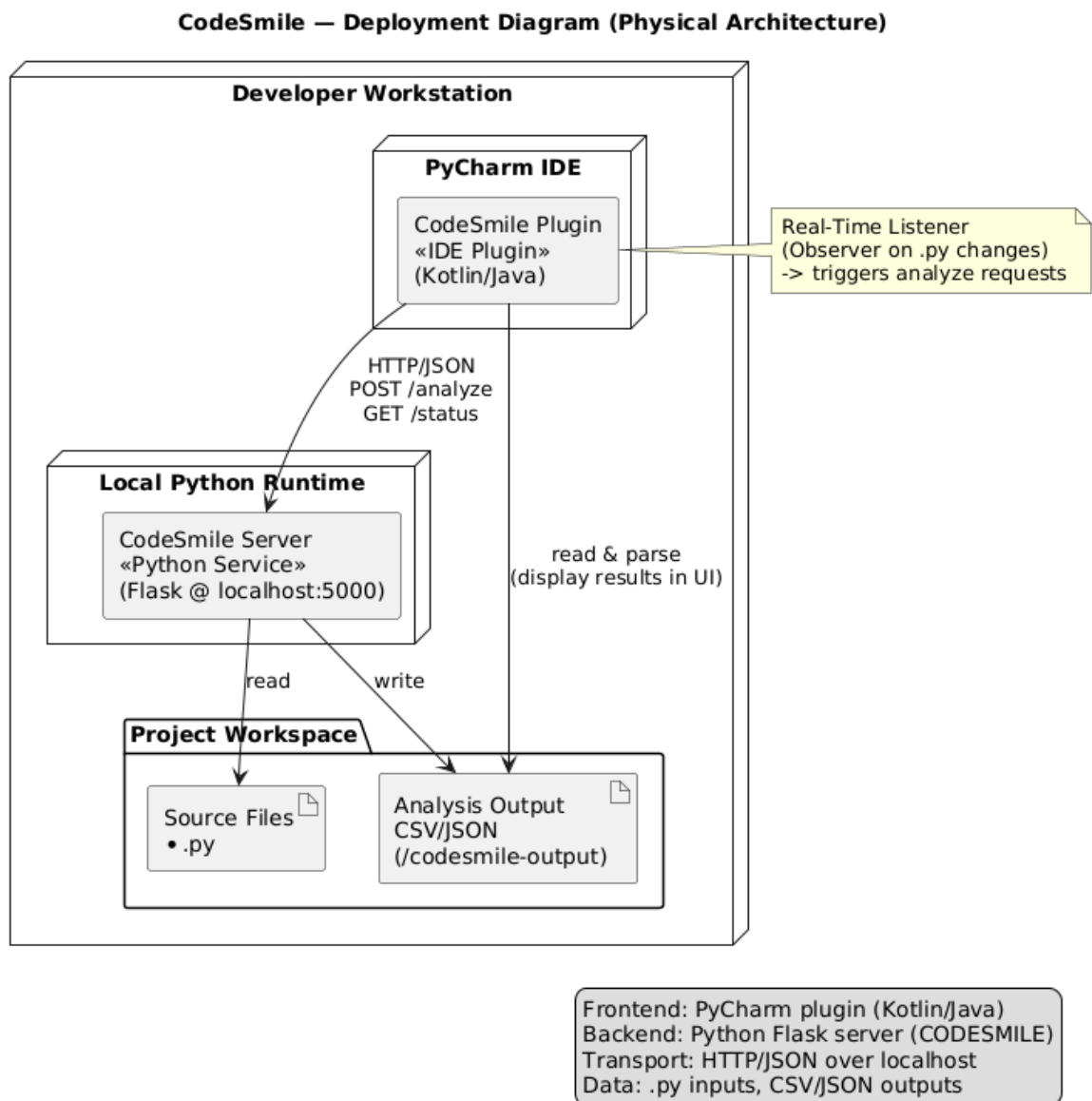


Figura 3.3: Architettura fisica: interazione tra il plugin (PyCharm) e il backend Python.

In sintesi, il diagramma evidenzia come l'utente interagisca con il plugin direttamente dall'IDE, come il plugin agisca da mediatore con il backend Python via

HTTP e come il server elabori i file sorgente locali producendo un output strutturato. Questa rappresentazione supporta la comprensione della distribuzione fisica dei componenti e dei canali di comunicazione utilizzati.

La Tabella 3.3 riassume le responsabilità principali dei sottosistemi e i requisiti che soddisfano:

Tabella 3.3: Sottosistemi, responsabilità e requisiti soddisfatti.

Sottosistema	Responsabilità principali	Requisiti soddisfatti
UI – Tool Window menu	Selezione modalità; visualizzazione risultati	FR2, FR4
Gestione Server Python	Avvio e monitoraggio backend	FR1, NFR1
NetworkUtil	Invio/ricezione richieste HTTP (Facade)	FR1, NFR1
ReportUtil	Formattazione output; pulizia file	FR4
Monitoraggio Real-Time	Listener per modifiche file; analisi automatica (Observer)	FR3, NFR1

3.4 Funzionalità principali

CodeSmile è stato progettato per offrire un insieme di funzionalità pensate per integrarsi in modo naturale nel flusso di lavoro dello sviluppatore, fornendo strumenti di rilevamento degli ML-CSs sia **on-demand** sia **in tempo reale**. L'obiettivo è ridurre il debito tecnico già in fase di sviluppo, migliorando la qualità del codice e prevenendo l'introduzione di *smell* che potrebbero compromettere la manutenibilità e le prestazioni del sistema [19].

Il plugin supporta quattro modalità operative principali, ciascuna legata a specifici requisiti funzionali, che sono le seguenti:

1. **Analisi dell'intero progetto (UC1):** esegue la scansione di tutti i file Python presenti nel progetto aperto in PyCharm. Utile per una revisione complessiva

della qualità del codice e per individuare *smell* diffusi in più file. (FR1)

2. **Analisi del file corrente (UC2):** analizza esclusivamente il file attualmente aperto nell'editor, riducendo i tempi di elaborazione quando si vuole verificare una porzione di codice specifica. (FR1, FR2)
3. **Analisi multipla (UC3):** consente di selezionare e analizzare un sottoinsieme di file all'interno del progetto. Utile quando si lavora su moduli specifici o durante la revisione del codice di un collega. (FR1, FR2)
4. **Analisi real-time (UC4):** attiva un listener che monitora in tempo reale i file Python aperti e rilancia automaticamente l'analisi a ogni modifica salvata. Permette di ricevere un feedback immediato, prevenendo la persistenza di *smell* nel codice. Implementa l'*Observer Pattern* e soddisfa FR3.

Le modalità UC1–UC4 sono accessibili dall'interfaccia principale del plugin (Tool Window Menu), che fornisce pulsanti e opzioni per selezionare la modalità e visualizzare i risultati. L'integrazione diretta nell'IDE riduce la necessità di strumenti esterni, favorendo un approccio di **continuous quality assurance** [18].

Tabella 3.4: Modalità operative di CodeSmile, caratteristiche e requisiti soddisfatti.

Modalità	Descrizione	Requisiti
UC1 – Analisi intero progetto	Scansione di tutti i file Python nel progetto aperto.	FR1
UC2 – Analisi file corrente	Analisi esclusiva del file aperto nell'editor.	FR1, FR2
UC3 – Analisi multipla	Analisi di un sottoinsieme di file selezionati.	FR1, FR2
UC4 – Analisi real-time	Monitoraggio continuo e analisi automatica a ogni modifica salvata.	FR3

3.5 Implementazione

L'implementazione di CodeSmile ha seguito i principi di progettazione e sviluppo descritti da Sommerville [16] e Bruegge & Dutoit [15], con particolare attenzione alla separazione dei moduli, all'uso di *design patterns* e alla scelta di tecnologie adatte all'integrazione con PyCharm.

Il sistema si compone di due macro-componenti implementativi:

- **Frontend** – il plugin integrato in PyCharm, sviluppato in Kotlin per garantire piena compatibilità con l'IntelliJ Platform e le API JetBrains.
- **Backend** – un server Python, sviluppato con Flask, che esegue il motore di analisi CODESMILE e produce un output strutturato (CSV o JSON) elaborato dal frontend.

Tecnologie adottate

- **Linguaggio frontend:** Kotlin, per integrazione nativa con IntelliJ e PyCharm.
- **Framework backend:** Flask (Python), per esporre CODESMILE come servizio REST.
- **Build system:** Gradle, per gestione delle dipendenze e packaging del plugin.
- **Formati di configurazione:** XML (metadati) e YAML (preferenze utente).
- **Formato output analisi:** CSV per output leggibile e facilmente processabile.

Struttura del codice

Il codice del plugin è organizzato in pacchetti che riflettono i sottosistemi descritti nella Sezione 3.3 e già rappresentati nel diagramma dei componenti (Figura 3.2):

- `ui` – gestione della Tool Window e interazione con l'utente.
- `network` – invio e ricezione delle richieste HTTP.
- `server` – gestione del ciclo di vita del backend Python.
- `utils` – formattazione dei risultati e funzioni di supporto.

3.6 Pattern architetturali e rilascio del software

La struttura di CodeSmile è stata progettata con attenzione alla riusabilità, modularità e manutenibilità, facendo riferimento alle best practice illustrate in letteratura [17, 20, 18]. In particolare, sono stati adottati i seguenti *Design Patterns*:

- **Observer**: utilizzato per implementare la rilevazione in tempo reale; il `RealTimeDetectionListener` monitora le modifiche ai file Python e notifica automaticamente il modulo di analisi per fornire feedback immediato.
- **Facade**: il modulo `NetworkUtil` funge da interfaccia semplificata verso il backend Python, nascondendo la complessità delle operazioni HTTP e del formato dei messaggi.
- **Builder/Formatter**: applicato nel modulo `ReportUtil` per comporre e formattare i risultati delle analisi in un formato leggibile e strutturato per l'utente, separando la logica di costruzione del report dalla sua rappresentazione.

L'utilizzo di questi pattern facilita la separazione dei compiti tra i componenti del sistema, riduce le dipendenze incrociate e semplifica le future estensioni dell'architettura.

Per favorire la riproducibilità del lavoro e la collaborazione con la comunità di ricerca e industriale, il codice sorgente del plugin CodeSmile è reso disponibile su GitHub:

`https://github.com/Daniy2/CodeSmile`

Il repository include il codice completo, le istruzioni per l'installazione, la configurazione dell'ambiente di sviluppo e una serie di esempi di utilizzo, in linea con le raccomandazioni per la ricerca riproducibile [21].

Esperimento di Valutazione

4.1 Introduzione

L'esperimento di valutazione rappresenta una fase cruciale di questo lavoro di tesi, in quanto consente di verificare in modo sistematico l'efficacia e l'utilità percepita del plugin *CodeSmile* nel supportare gli sviluppatori durante l'individuazione dei Machine Learning-specific Code smell (ML-CSs). Dopo aver definito i requisiti e implementato le funzionalità principali dello strumento, si è ritenuto necessario progettare e condurre uno *user study controllato*, finalizzato a raccogliere sia dati oggettivi (ad esempio tempi e accuratezza nel rilevamento degli smell) sia evidenze soggettive (fiducia, percezione di utilità e facilità d'uso del tool).

L'obiettivo principale dello studio è duplice: da un lato valutare se l'impiego del plugin comporti un miglioramento in termini di *efficacia* ed *efficienza* rispetto all'analisi manuale, dall'altro indagare il grado di *accettazione e soddisfazione* degli utenti verso lo strumento, con particolare attenzione all'effetto di apprendimento derivante dall'utilizzo.

La letteratura sottolinea come il design di uno user study richieda un approccio metodologicamente rigoroso, che tenga conto di obiettivi chiari, selezione dei partecipanti, criteri di raccolta dati e misurazione delle variabili di interesse. In ambito HCI,

testi di riferimento come *Research Methods in Human–Computer Interaction* di Lazar et al. [22] forniscono linee guida per la progettazione di esperimenti controllati e survey, evidenziando la necessità di garantire validità e replicabilità attraverso un’attenta pianificazione del campione e delle metriche da raccogliere. Parallelamente, studi su survey design in HCI hanno mostrato l’importanza di ridurre bias e ambiguità nei questionari, sottolineando come la qualità delle risposte dipenda fortemente dalla chiarezza delle domande e dalla struttura della scala di risposta [23].

Diversi studi hanno inoltre dimostrato il valore del *feedback qualitativo degli utenti* come complemento alle metriche quantitative, in quanto capace di evidenziare problemi di comprensione e aspetti di usabilità difficilmente rilevabili tramite soli dati numerici [24]. Altri lavori hanno posto attenzione al coinvolgimento degli stakeholder e alla necessità di integrare valutazioni in contesti realistici, mostrando come la qualità dei risultati dipenda anche dal grado di partecipazione e motivazione degli utenti coinvolti [25]. In parallelo, approcci alternativi come il *crowdsourcing* hanno messo in luce i vantaggi e le criticità di metodologie di valutazione distribuite, evidenziando come il rigore nel design dello studio sia essenziale per evitare bias e preservare la validità interna ed esterna dei risultati [26].

In coerenza con tali linee guida, lo user study presentato in questo capitolo è stato progettato come *esperimento controllato* con assegnazione bilanciata dei partecipanti a due gruppi (*Manuale* → *Tool* e *Tool* → *Manuale*), in modo da ridurre i possibili effetti di apprendimento legati all’ordine delle attività. La raccolta dati è stata condotta attraverso una combinazione di survey, task di detection e questionari comparativi, così da ottenere una valutazione complessiva che integra aspetti prestazionali e percettivi.

Le sezioni che seguono descrivono in dettaglio la struttura dello studio: dal *pre-screening survey*, utile a profilare i partecipanti e a definirne il background, alla fase di *testing & evaluation*, articolata in compiti di rilevamento manuale e con il supporto del tool, fino al *questionario comparativo e di feedback*, volto a raccogliere impressioni, preferenze e suggerimenti per sviluppi futuri.

4.2 Design dello Studio

Lo user study è stato concepito come *esperimento controllato* volto a confrontare due modalità di analisi del codice: manuale e con il supporto del plugin *CodeSmile*. Per ridurre possibili bias legati all'ordine delle attività, i partecipanti sono stati suddivisi in due gruppi bilanciati: il primo ha svolto inizialmente i compiti con il tool e successivamente in modalità manuale (gruppo *Tool* $\rightarrow$ *Manuale*), mentre il secondo ha seguito la sequenza inversa (*Manuale* $\rightarrow$ *Tool*). Questa scelta metodologica si ispira a pratiche consolidate nella progettazione di esperimenti comparativi in ambito HCI [22], dove il bilanciamento dell'ordine rappresenta una strategia efficace per limitare il learning effect o affaticamento.

Il design sperimentale si articola in tre fasi principali:

1. **Pre-Screening Survey**, finalizzata a raccogliere informazioni sul profilo dei partecipanti (ruolo, esperienza in ML, familiarità con librerie e conoscenza dei code smell).
2. **Testing & Evaluation**. In questa fase i partecipanti hanno svolto compiti di individuazione di ML-CSs sia manualmente che con il supporto del plugin, con un ordine bilanciato tra i gruppi per ridurre possibili bias di apprendimento. Ogni soggetto ha ricevuto un insieme di sei file Python preparati ad hoc (due set distinti: *s1-s6* e *m1-m6*), costruiti per rappresentare diversi scenari di rilevamento:

- *True Positive (TP)*: smell realmente presente e correttamente rilevato;
- *False Positive (FP)*: smell erroneamente segnalato dal tool;
- *False Negative (FN)*: smell realmente presente ma non rilevato;
- *True Negative (TN)*: file privi di smell, usati come distrattori.

Durante l'ispezione ogni file veniva analizzato seguendo una procedura standard:

- (a) Il partecipante riceveva il file Python e lo esaminava con o senza tool, a seconda della fase;

- (b) Compilava la sezione del survey relativa al file, indicando gli smell individuati (tramite checkbox), le linee di codice corrispondenti e il tempo impiegato per l'analisi;
- (c) Aggiungeva eventuali commenti qualitativi sull'esperienza di detection.

Inoltre, a seconda della modalità:

- **Analisi manuale:** veniva chiesto di valutare livello di difficoltà e fiducia nel proprio lavoro (Likert scale a 5 punti).
- **Analisi con tool:** i partecipanti esprimevano il grado di accordo con le segnalazioni del plugin, potevano contestarle e segnalare ulteriori smell non rilevati, oltre a giudicare la chiarezza dei messaggi e l'usabilità complessiva.

Questa fase ha così permesso di raccogliere dati sia oggettivi (tempi, accuratezza, distribuzione TP/FP/FN/TN) sia soggettivi (fiducia, difficoltà percepita, utilità del tool), fornendo la base per il confronto tra le due modalità di analisi.

3. **Comparison & Feedback Survey**, volta a raccogliere valutazioni comparative sull'esperienza con e senza tool, il grado di fiducia nelle segnalazioni, le preferenze individuali e i suggerimenti per miglioramenti futuri.

Il numero complessivo di partecipanti è stato scelto per garantire un minimo di variabilità nei profili e ottenere dati quantitativi e qualitativi sufficienti all'analisi. Sebbene la numerosità non sia paragonabile a quella di indagini su larga scala, essa risulta adeguata a un esperimento esplorativo di natura accademica, come suggerito in letteratura [23].

La raccolta dei dati è stata resa sistematica attraverso survey strutturate, task di detection e annotazioni guidate, permettendo di misurare sia aspetti **quantitativi** (tempi medi, accuratezza, precision, recall, F1-score) sia aspetti **qualitativi** (fiducia, difficoltà percepita, utilità del tool, preferenza della modalità di analisi). Questa combinazione riflette un approccio misto (*mixed-methods*), ampiamente raccomandato negli studi di usabilità e user experience [24], poiché consente di integrare la robustezza delle metriche oggettive con la ricchezza del feedback soggettivo.

La tabella 4.1 mostra lo schema di rotazione adottato per i partecipanti. Ogni soggetto ha svolto i task su due insiemi distinti di file Python: il primo set (`s1.py-s6.py`) conteneva esempi costruiti per rappresentare specifici casi di rilevamento (**True Positive**, **False Positive**, **False Negative**, **True Negative**), mentre il secondo set (`m1.py-m6.py`) conteneva analogamente file con uno o più smell o distrattori privi di violazioni. In questo modo, ogni partecipante si è confrontato sia con situazioni in cui il tool forniva una segnalazione corretta, sia con casi in cui produceva output incompleti o errati. La rotazione tra i due gruppi (*Tool* $\rightarrow$ *Manuale* e *Manuale* $\rightarrow$ *Tool*) ha garantito che tutti i partecipanti fossero esposti a entrambi i set, riducendo l'influenza dell'ordine delle fasi sulle prestazioni e sulle percezioni raccolte.

Tabella 4.1: Assegnazione dei partecipanti ai gruppi sperimentali

Partecipante	Fase 1	Fase 2	File Fase 1	File Fase 2
P1	Tool 1	Manuale 2	s1.py-s6.py	m1.py-m6.py
P2	Manuale 1	Tool 2	s1.py-s6.py	m1.py-m6.py
P3	Tool 2	Manuale 1	m1.py-m6.py	s1.py-s6.py
P4	Manuale 2	Tool 1	m1.py-m6.py	s1.py-s6.py
<i>Dal Partecipante P5 ricomincia la rotazione</i>				

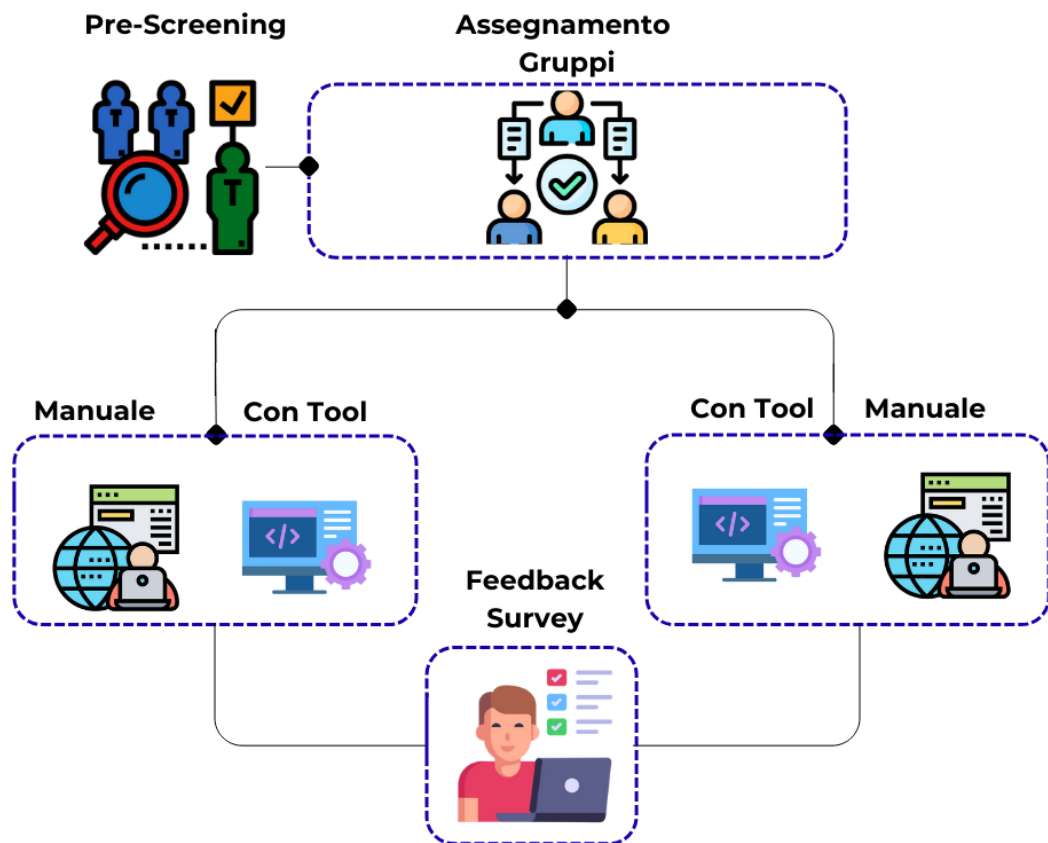


Figura 4.1: Schema del flusso sperimentale: Pre-Screening → Testing & Evaluation → Comparison & Feedback

Come mostrato in Figura 4.1, l'intero processo sperimentale è stato articolato in modo sequenziale e strutturato, seguendo un flusso che parte dalla fase preliminare di *pre-screening*, prosegue con le attività di *testing & evaluation* e si conclude con il *questionario comparativo e di feedback*. L'inclusione del flowchart tratto dal documento di design dello User Study consente di avere una visione sintetica e immediata del percorso svolto dai partecipanti, facilitando la comprensione del design sperimentale e rafforzandone la coerenza metodologica.

4.3 Pre-Screening Survey

La prima fase dello user study ha previsto la somministrazione di un questionario preliminare (*pre-screening survey*), con l'obiettivo di raccogliere informazioni di base sul profilo dei partecipanti e di mappare il loro livello di conoscenza ed

esperienza nel dominio del Machine Learning e dei code smell. Questa fase è risultata essenziale sia per comprendere il background dei soggetti coinvolti, sia per consentire un'assegnazione bilanciata nei due gruppi sperimentali (*Manuale* → *Tool* e *Tool* → *Manuale*).

Il questionario era strutturato in tre sezioni principali:

- **Anagrafica:** raccolta di dati sul ruolo occupato (es. studente, ricercatore, sviluppatore) e sugli anni di esperienza nel campo del Machine Learning, tramite domande a risposta multipla.
- **Conoscenza ed esperienza:** autovalutazione delle proprie competenze nelle diverse fasi di una pipeline ML (data preparation, modeling, training, evaluation, deployment) tramite una scala Likert a 5 punti. In questa sezione veniva anche richiesto di valutare la propria familiarità con alcune librerie Python comunemente impiegate in progetti di ML (Pandas, TensorFlow, PyTorch).
- **Conoscenza di code smell:** verifica preliminare del livello di consapevolezza dei partecipanti sull'argomento. In particolare, veniva chiesto se conoscessero strumenti di rilevamento di code smell tradizionali (es. SonarQube, flake8, pylint) e se avessero mai avuto esperienza con tool specifici per ML-CSs.

L'uso prevalente di scale Likert a 5 punti ha permesso di quantificare in modo uniforme percezioni e competenze dichiarate, garantendo confrontabilità tra i diversi partecipanti e facilitando successive analisi statistiche. Come suggerito dalla letteratura in HCI, infatti, l'impiego di scale standardizzate rappresenta una buona prassi negli studi di usabilità e user experience, poiché consente di ridurre ambiguità interpretative e di misurare in modo strutturato variabili soggettive come familiarità e sicurezza [23, 22, 27].

Complessivamente, il pre-screening ha fornito dati fondamentali sia per caratterizzare il campione, evidenziandone i limiti e i punti di forza, sia per predisporre una classificazione equilibrata dei partecipanti nei due gruppi di trattamento, così da assicurare maggiore robustezza metodologica all'esperimento.

4.4 Testing & Evaluation

La seconda fase dello user study ha previsto lo svolgimento dei task di detection in due condizioni: manuale e con il supporto del plugin *CodeSmile*. Come descritto in precedenza, l'ordine delle due modalità è stato bilanciato tra i gruppi per ridurre possibili bias legati all'apprendimento.

Per la sperimentazione sono stati predisposti due insiemi di file Python, ciascuno costruito per rappresentare specifici scenari di rilevamento. In questo modo i partecipanti si sono trovati ad affrontare casi in cui il tool forniva segnalazioni corrette (*True Positive* e *True Negative*) e altri in cui le predizioni risultavano parziali o errate (*False Positive* e *False Negative*).

Tabella 4.2: File utilizzati nella fase di *Testing & Evaluation* e categoria di riferimento

File	Smell	Descrizione	Categoria
s1.py	CIDX	Chain Indexing diretto su DataFrame	TP
s2.py	CDE	Colonne senza dtype esplicito	TP
s3.py	GNC	Mancanza di <code>zero_grad()</code> prima di <code>backward()</code>	TP
s4.py	IPA	Uso corretto di <code>drop()</code> ma segnalato come errato	FP
s5.py	PC + TA	Uso combinato di <code>.forward()</code> e <code>tf.constant()</code> non rilevati	FN
s6.py	—	Distrattori (uso corretto di API Pandas e Torch)	TN
m1.py	PC	Uso di <code>model.forward()</code> anziché <code>model(x)</code>	TP
m2.py	IPA	Uso di <code>drop()</code> senza assegnazione né <code>inplace</code>	TP
m3.py	TA	Concatenazione inefficiente di tensori su <code>tf.constant()</code>	TP

m4.py	CDE	Dati non ambigui ma segnalati come smell	FP
m5.py	CIDX + GNC	Chain Indexing e backward senza <code>zero_grad()</code>	FN
m6.py	—	Distrattori con API corrette	TN

Come evidenziato nella Tabella 4.2, la colonna *Smell* riporta la tipologia di ML-CS segnalata dal tool, mentre la colonna *Categoria* indica se la rilevazione fosse corretta o meno rispetto alla ground truth stabilita a priori. I casi **True Positive (TP)** rappresentano file in cui il tool ha correttamente identificato lo smell presente, mentre nei **False Positive (FP)** è stato segnalato uno smell inesistente (es. `s4.py`, `m4.py`). I **False Negative (FN)** corrispondono a smell realmente presenti ma non rilevati dal tool (`s5.py`, `m5.py`), lasciando quindi al partecipante la possibilità di individuarli manualmente. Infine, i **True Negative (TN)** sono file privi di smell, classificati in modo corretto dallo strumento (`s6.py`, `m6.py`).

Durante questa fase i partecipanti hanno dovuto:

- individuare e descrivere gli smell presenti nei file;
- indicarne le linee di codice corrispondenti;
- stimare difficoltà e fiducia nel proprio lavoro (modalità manuale);
- valutare il grado di accordo con le segnalazioni fornite dal tool (modalità con plugin).

La varietà di casi TP, FP, FN e TN ha permesso di misurare l'accuratezza e di osservare fenomeni cognitivi come l'*over-trust* [28] verso il tool o, al contrario, l'atteggiamento critico dei partecipanti.

La selezione dei sei smell non è stata casuale, ma si è basata sui risultati di Recupito et al. [5], che hanno evidenziato come *Chain Indexing (CIDX)*, *Columns and Datatype not Explicitly set (CDE)*, *Gradients Not Cleared Before Backward Propagation (GNC)*, *In-Place APIs Misused (IPA)*, *TensorArray Not Used (TA)* e *PyTorch Call Method Misused (PC)* figurino tra i ML-CSs più ricorrenti e persistenti nei progetti open

source. Limitare l'esperimento a tali smell ha quindi consentito di concentrarsi sui casi di maggiore rilevanza empirica, mantenendo al tempo stesso un carico cognitivo gestibile per i partecipanti e preservando la validità interna dello studio.

4.5 Comparison & Feedback Survey

La terza e ultima fase dello user study ha previsto la somministrazione di un questionario comparativo e di feedback, con l'obiettivo di raccogliere le percezioni dei partecipanti dopo aver sperimentato entrambe le modalità di analisi (manuale e con tool). Questa fase è centrale, poiché permette di integrare i dati oggettivi (tempi e accuratezza) con le valutazioni soggettive degli utenti, fornendo così le basi per rispondere alle **Research Questions (RQ)** definite in questo lavoro di tesi.

Il questionario è stato articolato in tre blocchi tematici: confronto tra modalità di analisi, valutazione dell'usabilità e utilità del tool, e supporto all'apprendimento. Sulla base di queste dimensioni sono state formulate le seguenti Research Questions.

Q RQ₁. *In che misura l'utilizzo di CodeSmile incrementa l'efficacia nella rilevazione di Machine Learning-specific code smell rispetto a una rilevazione manuale?*

Questa RQ indaga se il tool fornisce un supporto effettivo nell'individuazione degli smell, non solo in termini quantitativi (numero di smell rilevati), ma anche in termini di qualità e coerenza delle segnalazioni rispetto al codice analizzato.

Q RQ₂. *Quanto incide l'utilizzo di CodeSmile sulla riduzione del tempo necessario per la rilevazione di ML-specific code smell?*

Questa domanda punta a valutare l'efficienza del tool, verificando se e in che misura esso riduca l'impegno richiesto agli sviluppatori rispetto all'analisi manuale, con un'attenzione particolare al bilanciamento tra rapidità e accuratezza.

Q RQ₃. *Come viene percepita l'usabilità e l'utilità di CodeSmile dagli sviluppatori operanti in ambito Machine Learning?*

Qui si indaga la dimensione soggettiva: chiarezza dei messaggi, semplicità di interazione con l'interfaccia, utilità percepita del tool nel supportare il lavoro quotidiano e la propensione a consigliarne l'utilizzo ad altri colleghi o a integrarlo nei propri progetti.

Q RQ₄. *L'utilizzo di CodeSmile supporta l'apprendimento di concetti relativi ai code smell e accresce la percezione del rischio dei code smell nei sistemi ML-enabled?*

Infine, questa RQ esplora il potenziale *effetto di apprendimento* derivante dall'esperienza con il tool: se cioè l'uso del plugin abbia aiutato i partecipanti ad acquisire maggiore consapevolezza sulla pericolosità dei ML-CSs e a prestare più attenzione a tali aspetti nei propri progetti futuri.

Le quattro Research Questions così definite guideranno la discussione dei risultati presentata nel Capitolo 5, consentendo di valutare in maniera integrata l'efficacia, l'efficienza, l'usabilità e l'impatto formativo del tool.

4.6 Metriche di Analisi

Per valutare in modo completo l'impatto del plugin *CodeSmile*, nello user study sono state considerate sia metriche oggettive, derivate dalle prestazioni nei task, sia metriche soggettive, raccolte tramite i questionari di valutazione e feedback. Questa combinazione consente di integrare i dati numerici con la percezione degli utenti, in linea con le buone pratiche adottate negli studi di usabilità e user experience.

4.6.1 Metriche oggettive

Le metriche oggettive sono state ricavate direttamente dall'esecuzione dei task da parte dei partecipanti e comprendono:

- **Accuratezza (Accuracy):** percentuale complessiva di rilevamenti corretti (True Positive e True Negative) rispetto al totale dei casi.
- **Precision, Recall e F1-score:** misure standard per valutare la qualità della rilevazione, distinguendo tra capacità di individuare effettivamente gli smell presenti e riduzione dei falsi allarmi.
- **Tempi di esecuzione:** minuti impiegati per completare i task di analisi manuale e con il tool, utilizzati per confrontare l'efficienza delle due modalità.

4.6.2 Metriche soggettive

Le metriche soggettive derivano invece dalle risposte fornite nei survey, prevalentemente tramite scale Likert a 5 punti e domande aperte. Esse includono:

- **Fiducia nei risultati del tool:** grado di accordo con le segnalazioni prodotte e percezione di affidabilità dello strumento.
- **Usabilità e chiarezza:** giudizi sulla comprensibilità dei messaggi e sulla facilità di interazione con l'interfaccia del plugin.
- **Utilità percepita:** valutazione del contributo del tool nel supportare l'attività di detection, anche in termini di adozione futura o raccomandazione a colleghi.
- **Effetto di apprendimento:** percezione di aver acquisito nuove conoscenze sui code smell e di una maggiore consapevolezza circa la loro pericolosità e frequenza nei sistemi ML-enabled.

L'analisi combinata di queste metriche consente di fornire una valutazione completa del tool, misurandone l'efficacia, l'efficienza, la qualità dell'esperienza d'uso e il potenziale contributo formativo. I risultati relativi a ciascuna metrica verranno presentati e discussi nel Capitolo 5.

4.7 Sintesi del Design

Il design sperimentale presentato in questo capitolo ha seguito un approccio strutturato e progressivo, articolato in tre fasi principali: *pre-screening survey*, *testing & evaluation* e *comparison & feedback survey*.

La prima fase ha permesso di raccogliere informazioni sul profilo dei partecipanti e di garantirne un'assegnazione bilanciata nei due gruppi sperimentali, riducendo possibili bias legati all'ordine delle attività. La seconda fase ha consentito di confrontare in modo controllato l'analisi manuale e quella supportata dal tool, esponendo i partecipanti a scenari eterogenei (True Positive, False Positive, False Negative, True Negative). Infine, la terza fase ha raccolto le percezioni soggettive sull'efficacia, l'efficienza, l'usabilità e il potenziale contributo formativo del plugin, fornendo le basi per rispondere alle Research Questions.

Complessivamente, il design sperimentale ha combinato metriche oggettive e soggettive, consentendo di valutare CodeSmile da prospettive complementari. Tale impostazione permette non solo di misurare prestazioni numeriche, ma anche di comprendere l'esperienza degli utenti e il valore aggiunto del tool in un contesto realistico.

L'insieme delle informazioni raccolte costituisce dunque la base per l'analisi e la discussione dei risultati che verranno presentati nel Capitolo 5.

Analisi dei Risultati

5.1 Introduzione

Per garantire una lettura chiara e progressiva, la discussione è organizzata in sezioni tematiche che seguono l'ordine dei grafici e delle analisi introdotte nei documenti sperimentali: si parte dal profilo dei partecipanti, utile a contestualizzare i risultati, per poi affrontare le metriche oggettive relative all'esperimento, l'analisi dettagliata per tipologia di smell, la percezione soggettiva di pericolosità e frequenza, fino ad arrivare agli effetti di apprendimento e al giudizio complessivo sul tool. Il capitolo si conclude con una discussione integrata, in cui i risultati vengono messi in relazione diretta con le quattro Research Questions, fornendo così evidenze empiriche sull'impatto che strumenti come CodeSmile possono avere nel migliorare la qualità del software basato su Machine Learning e nel ridurre fenomeni di debito tecnico associati ai ML-CSs.

5.2 Profilo dei Partecipanti

Prima di analizzare i risultati sperimentali, è opportuno delineare il profilo dei partecipanti coinvolti nello user study, al fine di contestualizzare meglio i dati raccolti.

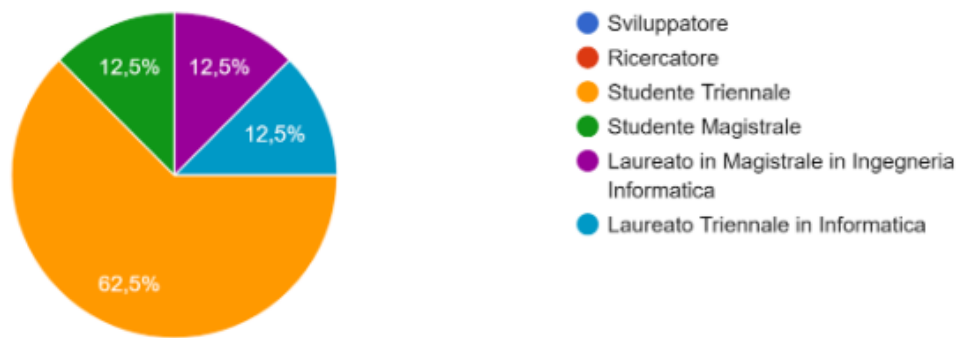
Dal punto di vista anagrafico, il campione era composto prevalentemente da studenti di corsi di laurea triennale in informatica, con poche eccezioni rappresentate da laureati magistrali o figure con maggiore esperienza professionale. La distribuzione dei ruoli (Figura 5.1) riflette quindi un campione in larga parte formato da sviluppatori alle prime esperienze, fattore che influenza direttamente la capacità di individuare correttamente i *Machine Learning-specific Code smell* (ML-CSs).

L'esperienza complessiva in ambito Machine Learning è risultata generalmente limitata: la maggioranza dei partecipanti ha dichiarato meno di un anno di esperienza, con solo pochi casi superiori ai due anni. Analizzando le diverse fasi della pipeline ML (Figura 5.2), emerge come la **Data Preparation** rappresenti la fase meglio conosciuta (con un livello dichiarato tra discreto e buono dal 75% dei rispondenti), seguita dal **Modeling** (50%). Al contrario, le fasi di **Evaluation** e soprattutto di **Deployment** hanno evidenziato conoscenze carenti: nel secondo caso oltre il 60% dei partecipanti si è auto-valutato con un livello scarso o molto scarso.

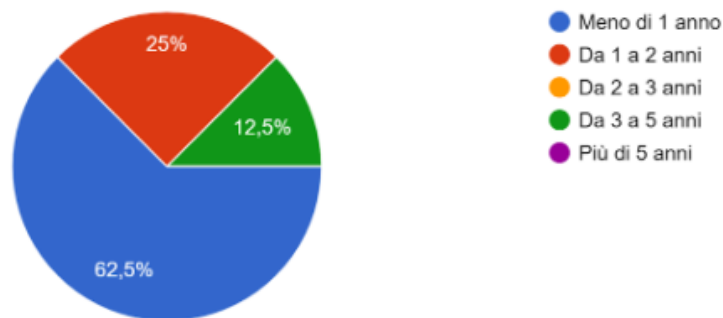
Per quanto riguarda la familiarità con le librerie Python più diffuse (Figura 5.3), **Pandas** è risultata la più conosciuta, con quasi il 40% dei partecipanti che ne ha dichiarato un livello alto o molto alto. **TensorFlow** si colloca in una posizione intermedia, con livelli prevalentemente bassi o discreti, mentre **PyTorch** è emersa come la libreria meno familiare: il 75% dei partecipanti ha infatti dichiarato una conoscenza molto bassa o bassa.

Sul fronte della conoscenza dei code smell (Figura 5.4), il livello medio è risultato complessivamente **scarso**: il 50% dei partecipanti ha dichiarato di non conoscere affatto i ML-CSs e nessuno si è dichiarato “molto esperto” in materia. Analogamente, il 75% non aveva mai utilizzato strumenti di rilevamento di code smell tradizionali (come SonarQube, flake8 o pylint), mentre nessuno conosceva tool specifici per ML-CSs (Figura 5.5).

Questi dati di profilazione confermano come il campione fosse costituito da sviluppatori in gran parte **inesperti** rispetto ai code smell in ambito ML. Ciò fornisce un importante contesto interpretativo: i risultati dell'esperimento vanno letti anche alla luce di questa bassa familiarità, che da un lato può aver reso più evidente l'utilità del tool, dall'altro può aver accentuato fenomeni cognitivi come l'*over-trust* verso l'analisi automatica.



(a) Distribuzione dei ruoli dei partecipanti



(b) Anni di esperienza in ambito ML

Figura 5.1: Profilo anagrafico dei partecipanti

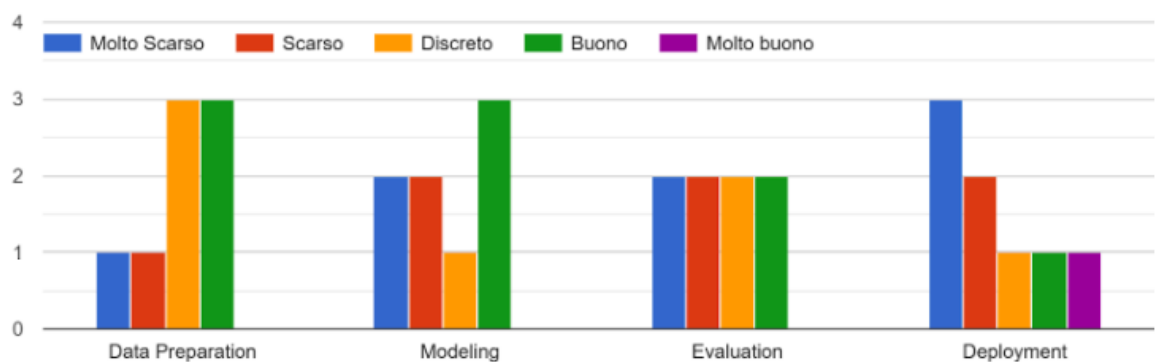


Figura 5.2: Autovalutazione della conoscenza delle fasi della pipeline ML

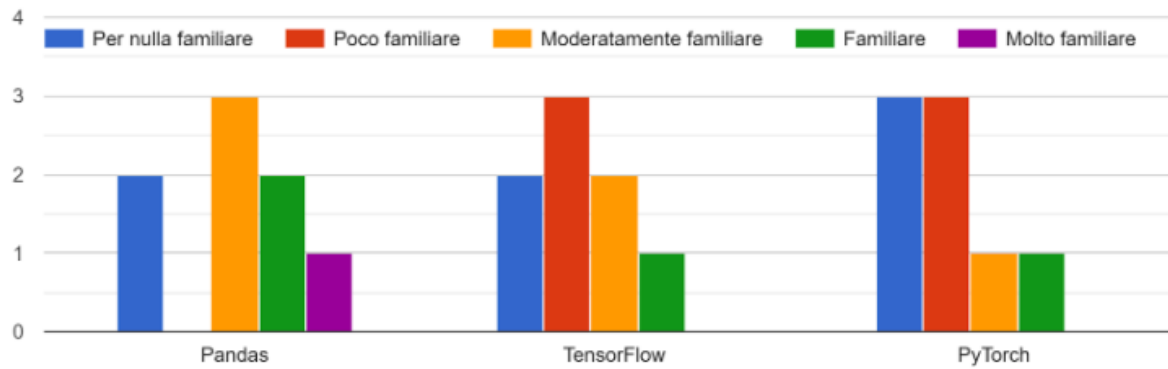


Figura 5.3: Familiarità dichiarata con le principali librerie Python per ML

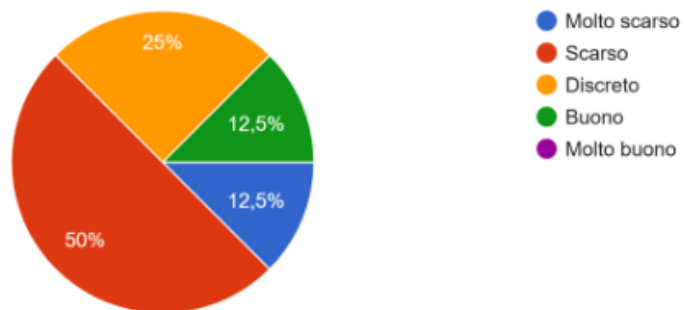


Figura 5.4: Livello di conoscenza dei code smell in ambito ML

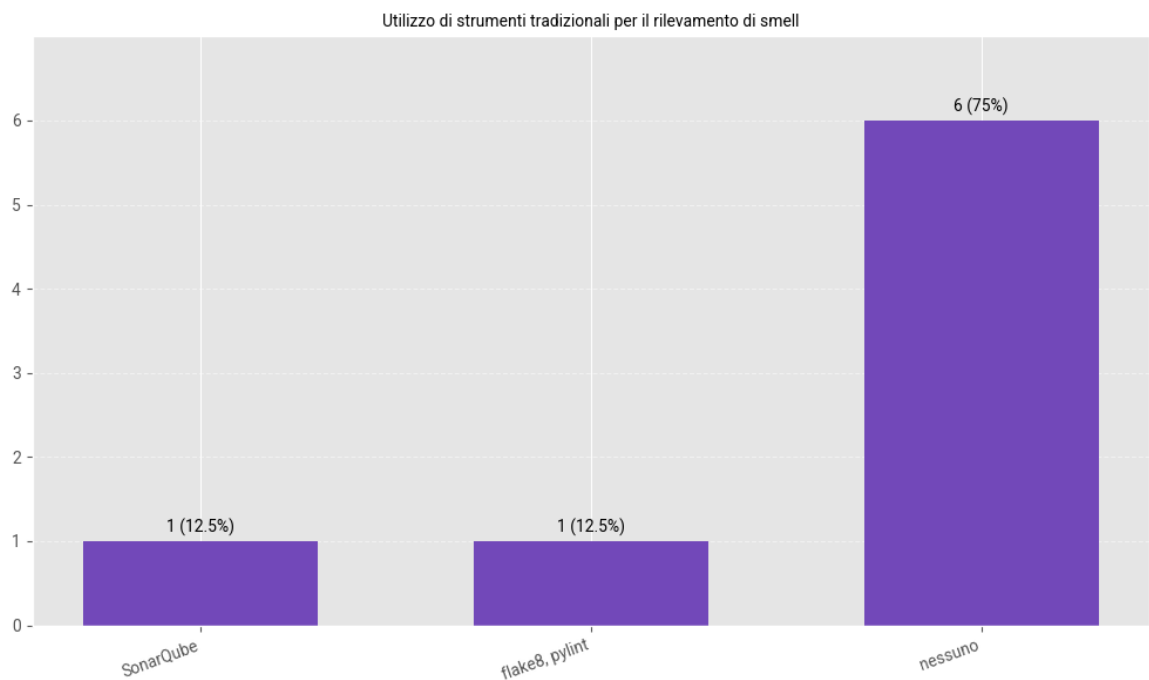


Figura 5.5: Conoscenza di strumenti di rilevamento di code smell

5.3 Metriche Oggettive dell'Esperimento

Per valutare in maniera quantitativa l'impatto dell'utilizzo del plugin *CodeSmile*, sono state considerate tre dimensioni principali:

1. l'efficacia della detection (confronto manuale vs tool);
2. i tempi medi di esecuzione dei compiti;
3. il livello di fiducia riposta nello strumento e il comportamento dei partecipanti di fronte a TP, TN, FP e FN.

5.3.1 Confronto complessivo Manuale vs Tool

Prima di analizzare i risultati per singola tipologia di smell, è utile proporre un confronto complessivo tra le due modalità di sperimentazione (manuale e con il supporto del tool). Questa analisi fornisce una visione d'insieme sull'efficacia del plugin, mostrando come le prestazioni globali dei partecipanti cambino a seconda dello scenario.

Nota metodologica Come descritto nel Capitolo 4 (Testing & Evaluation), lo user study è stato progettato come esperimento controllato con due gruppi bilanciati (*Tool* → *Manuale* e *Manuale* → *Tool*), al fine di ridurre possibili bias dovuti all'ordine delle attività. In ciascuna fase, ogni partecipante ha analizzato un set di sei file Python costruiti ad hoc per rappresentare diversi scenari di rilevamento: tre con smell effettivamente presenti (True Positive), uno privo di smell (True Negative), uno con smell erroneamente segnalato dal tool (False Positive) e uno con smell presenti ma non rilevati dal tool (False Negative).

Per costruire i risultati riportati in questa sezione è stato necessario definire un criterio uniforme di valutazione dei compiti svolti dai partecipanti. Le due fasi sperimentali erano bilanciate: in ciascuna modalità (manuale e con tool) i soggetti analizzavano 6 file, di cui 3 contenenti effettivamente uno smell (True Positive attesi), 1 privo di smell (True Negative), 1 file segnalato erroneamente dal tool come contenente uno smell (False Positive) e 1 file con smell presenti ma non rilevati dal tool (False Negative).

Per ogni file e partecipante è stato quindi attribuito un punteggio di correttezza compreso tra 0 e 1 secondo i criteri descritti nel capitolo sullo user study:

- **TP/TN:** punteggio 1 se il partecipante riconosceva correttamente la presenza/assenza di smell, 0 altrimenti;
- **FP:** punteggio 1 se si accorgeva dell'errore (non accettando la segnalazione scorretta), 0 se si allineava alla predizione del tool;
- **FN:** punteggio 1 se rilevava tutti gli smell mancanti, 0.5 se solo in parte, 0.25 se segnalava smell sbagliati, 0 se si allineava alla mancata segnalazione del tool.

Aggregando questi punteggi per categoria (TP, TN, FP, FN) e calcolandone la media sui partecipanti, si ottengono le percentuali mostrate in Figura 5.6. A partire da tali valori è stato poi possibile derivare le metriche complessive (Accuracy, Precision, Recall, Specificità, F1-score) riportate in Tabella 5.1.

Risultati complessivi La Figura 5.6 riporta un confronto diretto sulle quattro categorie di esito (True Positive, True Negative, False Positive, False Negative). Il grafico evidenzia un netto miglioramento nell'individuazione corretta degli smell realmente presenti (True Positive: 94.3% con il tool vs 75.8% manuale) e nella capacità di riconoscere correttamente i file privi di smell (Specificity: 100% vs 62.5%). D'altro canto, i partecipanti hanno mostrato una tendenza all'*over-trust* nei confronti del tool, accettandone le predizioni anche in presenza di falsi positivi e falsi negativi.

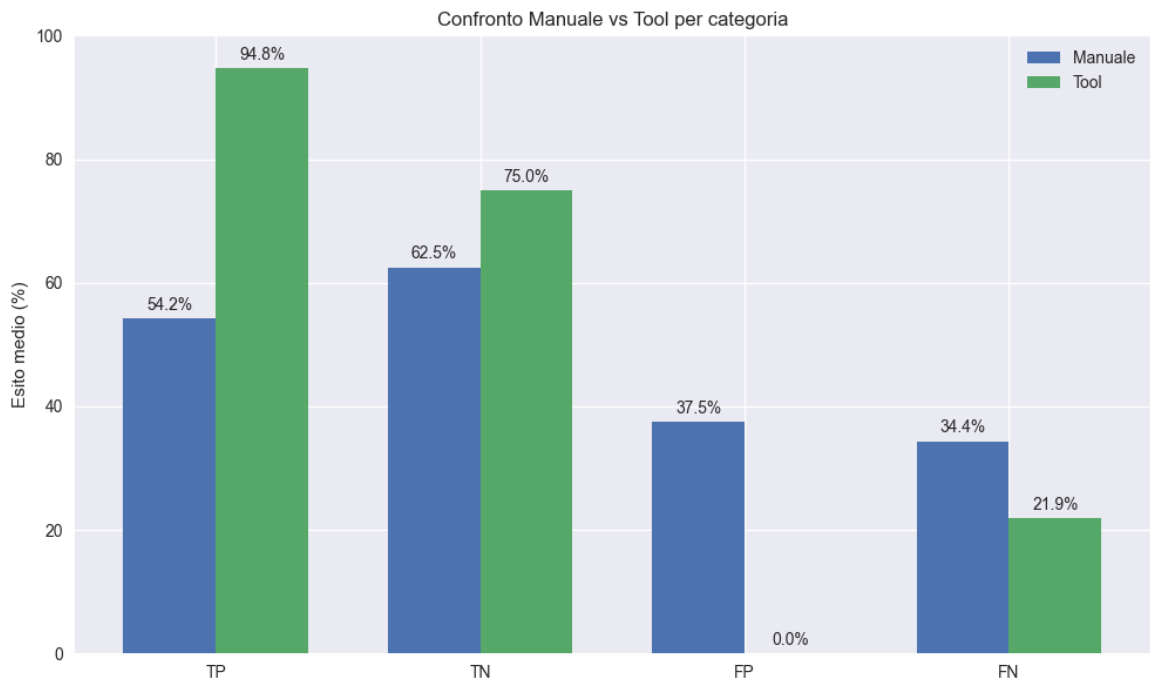


Figura 5.6: Confronto complessivo tra analisi manuale e con tool per categoria di esito

Per rendere più chiara la differenza tra le due modalità, la Tabella 5.1 riassume le principali metriche di classificazione: **accuracy**, **precision**, **recall**, **specificity** e **F1-score**. I valori confermano un miglioramento significativo in tutte le metriche con l'utilizzo del tool, con un incremento dell'**accuracy** dal 75.8% al 94.3% e un **F1-score** che passa da 81.9% a 96.3%. Questi dati evidenziano il guadagno complessivo in termini di efficacia, pur a fronte del rischio di una fiducia eccessiva verso l'automazione.

Tabella 5.1: Metriche complessive di detection in modalità manuale e con il tool

Modalità	Accuracy	Precision	Recall	Specificity	F1-score
Manuale	0.758	0.812	0.825	0.625	0.819
Tool	0.943	1.000	0.929	1.000	0.963

5.3.2 Tempi di analisi

Il boxplot dei tempi manuali (Figura 5.7) mostra una distribuzione molto concentrata attorno ai 4 minuti medi per file, con bassa variabilità tra i partecipanti. Per

quanto riguarda i tempi con il tool (Figura 5.8), i risultati evidenziano una drastica riduzione, prossima allo zero nella quasi totalità dei casi. Questi dati confermano come l'impiego del plugin azzeri di fatto il costo temporale dell'ispezione, rendendo l'attività molto più rapida rispetto all'analisi manuale.

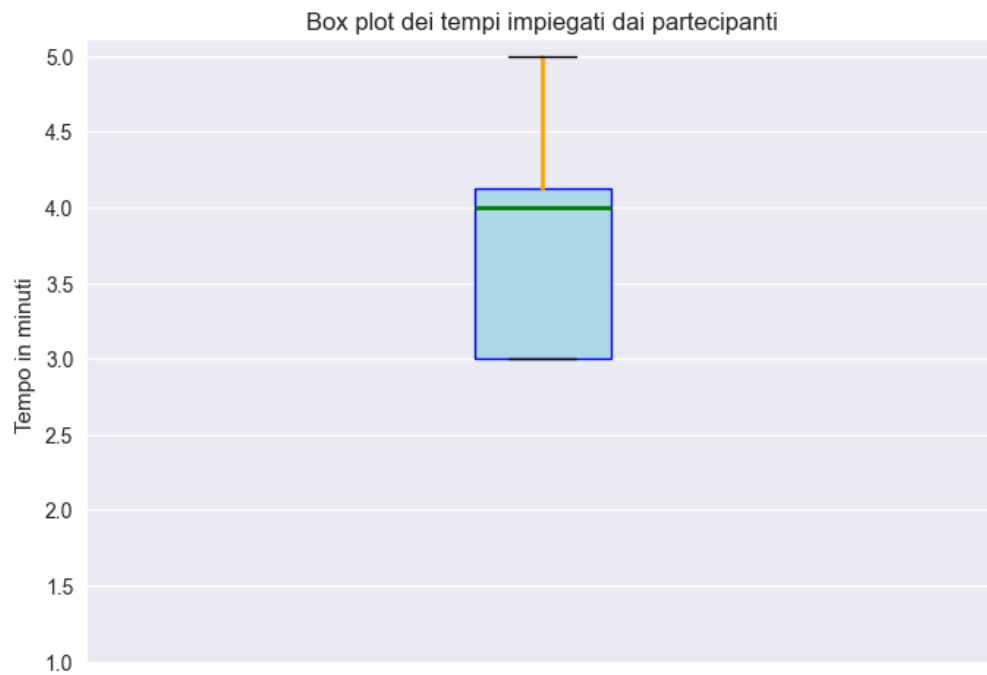


Figura 5.7: Boxplot dei tempi medi di analisi manuale



Figura 5.8: Boxplot dei tempi medi di analisi con il tool

5.3.3 Fiducia e comportamento per categoria di file

Un ulteriore aspetto considerato riguarda la fiducia riposta nel tool. Il boxplot dedicato (Figura 5.9) mostra valori mediamente alti, con mediana prossima a 4 su scala Likert a 5 punti e bassa dispersione. Questo suggerisce che i partecipanti abbiano percepito lo strumento come affidabile in modo relativamente uniforme. Tuttavia, un livello di fiducia così elevato, unito alla scarsa esperienza emersa nel profilo dei partecipanti, può essere interpretato anche come un indizio di *over-trust*, ossia la tendenza ad accettare passivamente l'analisi automatica senza metterla in discussione.

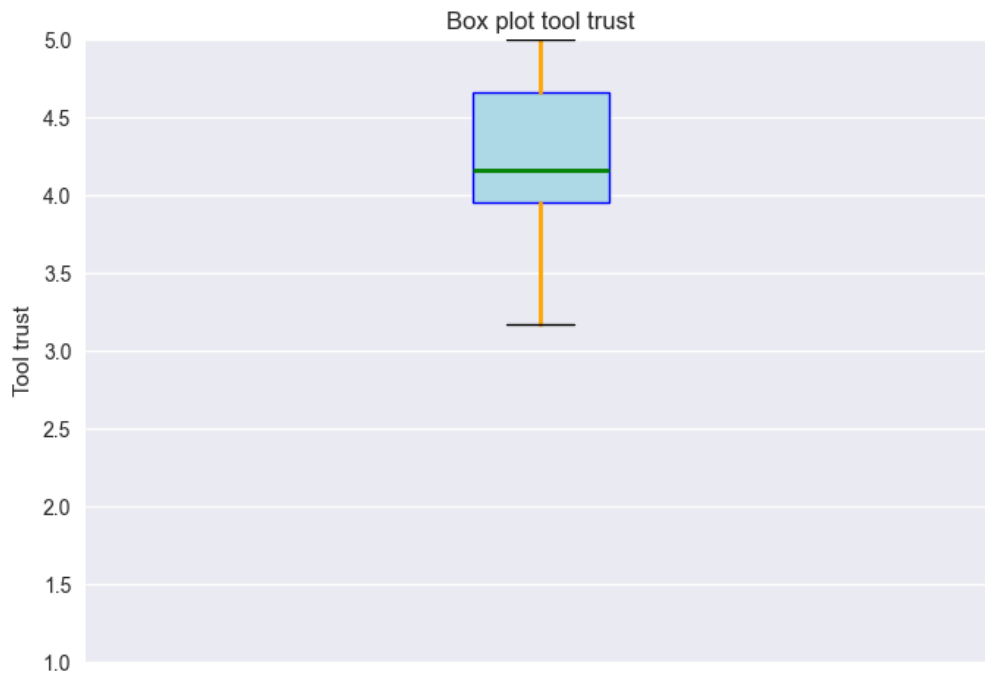


Figura 5.9: Boxplot della fiducia riposta nel tool da parte dei partecipanti

Per comprendere meglio dove questa fiducia si sia tradotta in benefici o in criticità, la Tabella 5.2 riporta i valori percentuali di rilevamento corretto per categoria di file (TP, TN, FP, FN) nelle due modalità. Questi dati, già rappresentati visivamente in Figura 5.6, permettono una lettura più dettagliata del fenomeno di *over-trust*.

Tabella 5.2: Percentuali di rilevamento corretto per categoria di file (Manuale vs Tool)

Categoria	Manuale	Tool	Differenza
True Positive	81.2%	94.3%	+13.1%
True Negative	62.5%	100%	+37.5%
False Positive	37.5%	0%	-37.5%
False Negative	34.4%	21.9%	-12.5%

Discussione Dalla tabella emergono alcuni pattern significativi:

- **True Positive e True Negative:** con il tool i partecipanti hanno mostrato prestazioni migliori, segno che la fiducia riposta nello strumento ha avuto un impatto positivo quando le sue predizioni erano corrette.
- **False Positive:** nell'analisi manuale oltre un terzo dei partecipanti (37.5%) ha riconosciuto l'errore e non ha accettato la segnalazione sbagliata. Con il tool questa capacità critica si azzerava completamente (0%), evidenziando un chiaro caso di *over-trust*.
- **False Negative:** anche in questo scenario la modalità manuale ha ottenuto risultati migliori (34.4% vs 21.9%), mostrando che i partecipanti hanno avuto più difficoltà a contestare le mancate segnalazioni del tool.

Questi risultati confermano che l'elevata fiducia complessiva (Figura 5.9) si traduce in un comportamento duale: da un lato migliora l'accuratezza su TP e TN, dall'altro riduce la capacità di mettere in discussione le predizioni errate o incomplete, accentuando il rischio di *over-trust* soprattutto nei casi di FP e FN.

5.4 Analisi per Tipologia di Smell

Un'analisi più dettagliata è stata condotta valutando le performance dei partecipanti in funzione delle diverse tipologie di *Machine Learning-specific Code smell*. A questo scopo sono state costruite le confusion matrix relative alle due modalità sperimentali (manuale e con tool), dalle quali sono state calcolate le metriche standard di precision, recall e F1-score.

Nel caso dell'**analisi manuale**, i risultati mostrano una generale difficoltà nel riconoscere correttamente gli smell. In Figura 5.10 è riportata la confusion matrix relativa a questa modalità, che permette di osservare la distribuzione dei rilevamenti per ogni categoria. Dalla matrice si evince una certa dispersione, con numerosi errori di classificazione e forti differenze tra smell.

I valori numerici associati a precision, recall e F1-score sono riportati nella Tabella 5.3. In particolare, lo smell **CDE (Columns and Datatype not Explicitly set)** si è rivelato particolarmente critico, con una precision molto bassa (0.125) e una

recall pari a 0.25, a testimonianza delle difficoltà incontrate dai partecipanti nell'individuare correttamente. Analogamente, anche lo smell **PC (PyTorch Call Method Misused)** ha evidenziato una forte asimmetria: se da un lato la precision ha raggiunto il valore massimo (1.0), dall'altro la recall si è fermata a 0.222, indicando che gran parte delle istanze non sono state rilevate. Risultati più bilanciati emergono invece per lo smell **TA (TensorArray not Used)**, con precision e recall entrambe pari a 0.667, mentre gli smell **CIDX**, **GNC** e **IPA** hanno registrato performance intermedie ma comunque lontane da valori soddisfacenti. In generale, l'accuratezza complessiva dell'analisi manuale si è attestata su valori poco sotto il 50%, coerentemente con la bassa esperienza dei partecipanti già evidenziata nella sezione di pre-screening.

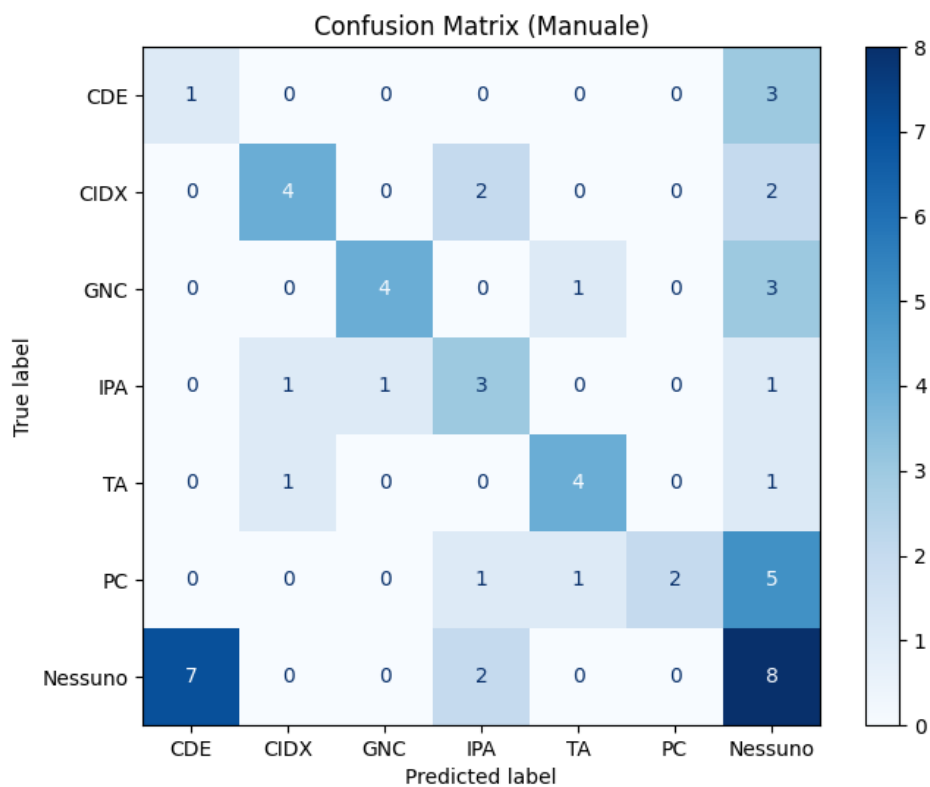


Figura 5.10: Confusion matrix relativa all'analisi manuale degli smell

Tabella 5.3: Metriche di precision, recall e F1-score per smell (analisi manuale)

Classe	Precision	Recall	F1-score
CDE	0.125	0.250	0.167
CIDX	0.667	0.500	0.571
GNC	0.800	0.500	0.615
IPA	0.375	0.500	0.429
TA	0.667	0.667	0.667
PC	1.000	0.222	0.364
Nessuno	0.348	0.471	0.400
Micro Avg	0.448	0.448	0.448
Macro Avg	0.569	0.444	0.459
Weighted Avg	0.576	0.448	0.462
Accuracy	0.448	-	-

Diverso il quadro nell'**analisi con tool**. La confusion matrix riportata in Figura 5.11 mostra una distribuzione più bilanciata rispetto a quella manuale, con un miglioramento evidente in diverse categorie di smell. I valori numerici delle metriche (Tabella 5.4) confermano questo andamento: ad esempio, **CIDX e IPA** hanno registrato precision e recall più solide rispetto alla fase manuale, a conferma della capacità del plugin di supportare i partecipanti in compiti altrimenti complessi.

Restano tuttavia alcune criticità: per lo smell **CDE**, pur essendo migliorata la recall (1.0), la precision è rimasta molto bassa (0.364), segnalando un'alta incidenza di falsi positivi accettati dai partecipanti. Anche la categoria **nessuno smell** ha riportato valori deboli (precision 0.353 e recall 0.333), probabilmente dovuti all'effetto di *over-trust* verso il tool, che ha portato i partecipanti ad accettare l'assenza di segnalazioni senza ulteriori verifiche.

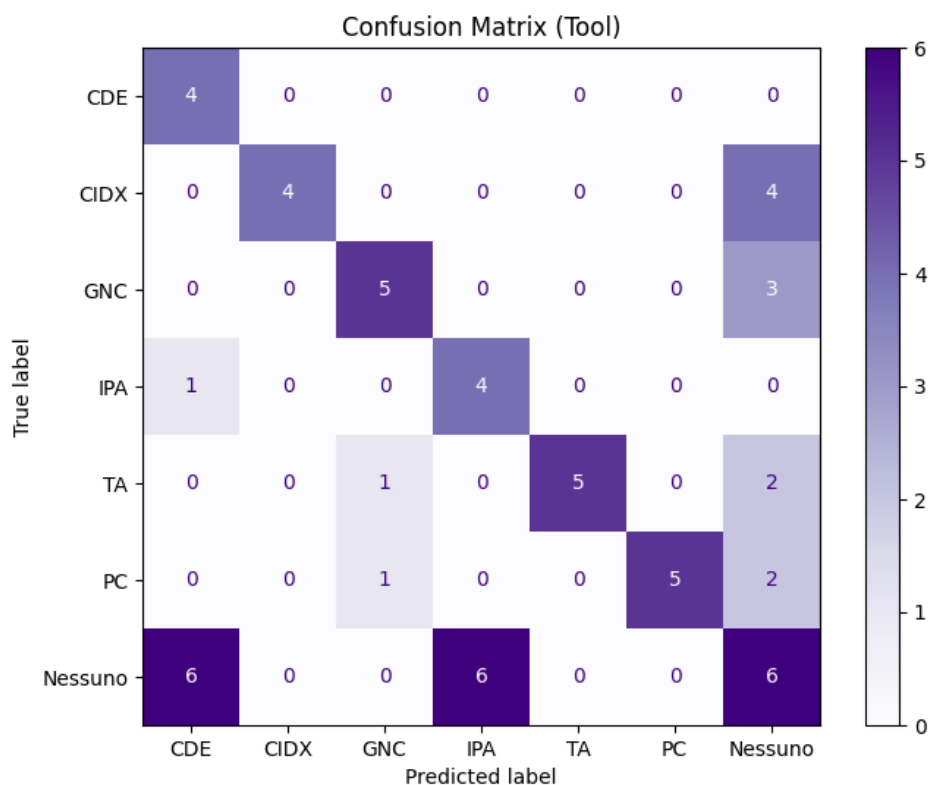


Figura 5.11: Confusion matrix relativa all'analisi con il tool

Tabella 5.4: Metriche di precision, recall e F1-score per smell (analisi con tool)

Classe	Precision	Recall	F1-score
CDE	0.364	1.000	0.533
CIDX	1.000	0.500	0.667
GNC	0.714	0.625	0.667
IPA	0.400	0.800	0.533
TA	1.000	0.625	0.769
PC	1.000	0.625	0.769
Nessuno	0.353	0.333	0.343
Micro Avg	0.559	0.559	0.559
Macro Avg	0.690	0.644	0.612

Weighted Avg	0.670	0.559	0.575
Accuracy	0.559	-	-

Complessivamente, l'uso del tool ha contribuito ad aumentare la consistenza delle rilevazioni, riducendo la variabilità e migliorando le performance rispetto all'analisi manuale. Tuttavia, il fenomeno dell'*over-trust* rimane evidente: i partecipanti, di fronte a predizioni errate o mancanti, hanno raramente esercitato spirito critico, confermando così uno dei rischi più discussi in letteratura rispetto all'adozione di strumenti di analisi automatica [22, 5].

5.5 Percezione di Pericolosità e Frequenza degli smell

Oltre alle metriche di performance, l'esperimento ha raccolto anche le percezioni soggettive dei partecipanti riguardo alla pericolosità e alla frequenza dei *Machine Learning-specific Code smell*. Queste valutazioni sono importanti perché permettono di comprendere non solo l'efficacia del tool, ma anche l'impatto formativo in termini di consapevolezza critica degli smell.

Per quanto riguarda la **pericolosità percepita**, i boxplot (Figura 5.12) mostrano che tutti gli smell sono stati considerati almeno moderatamente rischiosi, con mediane sempre uguali o superiori a 3 nella scala Likert utilizzata. Tra i diversi smell, **IPA (In-Place APIs Misused)** è stato percepito come il più pericoloso, con una mediana pari a 4 e una distribuzione relativamente compatta. Al contrario, gli smell **CDE (Columns and Datatype not Explicitly set)** e **TA (TensorArray not Used)** hanno riportato valori più bassi, con mediane intorno a 3, segnalando che i partecipanti li percepiscono come problematici ma non prioritari.

Sul fronte della **frequenza percepita**, i boxplot (Figura 5.13) evidenziano valori sensibilmente più bassi: quasi tutti gli smell hanno mediane comprese tra 1 e 2, a indicare che i partecipanti ritengono di averli incontrati raramente nella loro esperienza. Anche in questo caso la dispersione è minima, suggerendo una percezione condivisa di scarsa esposizione. L'unico caso con qualche variazione maggiore riguarda ancora **IPA**, che alcuni partecipanti hanno dichiarato di aver incontrato più spesso.

Complessivamente, i risultati mostrano un fenomeno già segnalato in letteratura [5]: gli sviluppatori tendono a percepire alcuni smell come molto pericolosi anche se dichiarano di non averli mai incontrati frequentemente, segnalando una conoscenza più teorica che esperienziale. Questo scollamento è particolarmente rilevante perché suggerisce che strumenti come *CodeSmile* possano avere anche una funzione educativa, colmando il divario tra consapevolezza astratta e pratica reale.

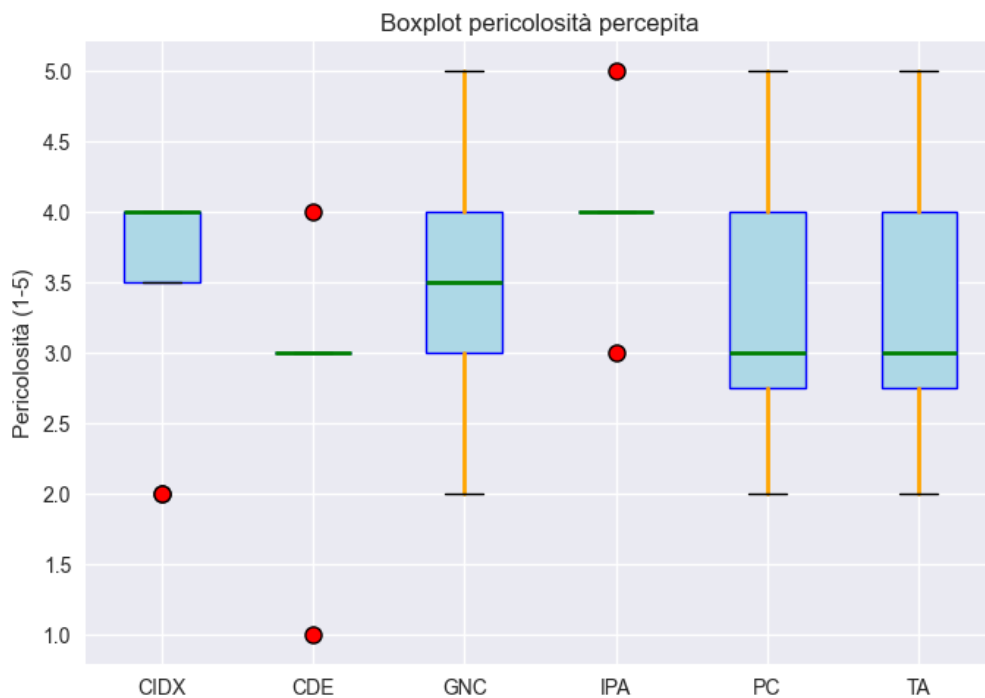


Figura 5.12: Boxplot della pericolosità percepita dei diversi ML-CSs

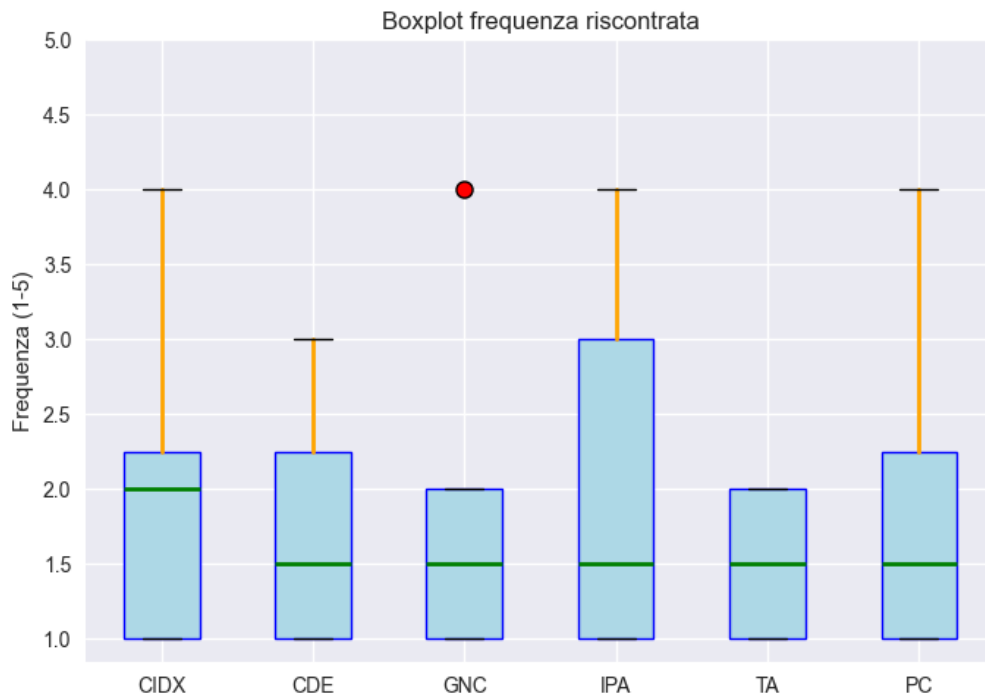


Figura 5.13: Boxplot della frequenza percepita dei diversi ML-CSs

5.6 Consapevolezza ed Effetti Formativi

Un aspetto centrale dello *user study* riguardava la capacità del tool non solo di supportare i partecipanti nei compiti di detection, ma anche di accrescere la loro consapevolezza sugli smell e sul loro impatto. Non si tratta, in questo contesto, del *learning effect* inteso come bias sperimentale legato all'ordine delle fasi (mitigato tramite bilanciamento dei gruppi), bensì del contributo formativo che l'esperimento ha avuto sugli sviluppatori coinvolti.

Il primo dato utile emerge dalla domanda relativa a quali smell i partecipanti avrebbero prestato maggiore attenzione in futuro. I risultati (Figura 5.14) evidenziano che lo smell IPA (In-Place APIs Misused) è stato indicato come prioritario dal 29.4% dei rispondenti, in linea con quanto già emerso nei boxplot di pericolosità percepita, dove IPA era stato valutato come il più critico. Al contrario, PC (PyTorch Call Method Misused) non è mai stato selezionato come smell degno di particolare attenzione, coerentemente con la percezione di bassa pericolosità.

Relativamente all'**outcome di apprendimento percepito**, i partecipanti hanno dichiarato di aver tratto un beneficio dall'esperimento sotto diversi profili (Figura 5.15). Le categorie più frequenti sono state: maggiore attenzione agli smell (38%), consapevolezza dei rischi connessi (38%) e scoperta di smell fino a quel momento sconosciuti (25%). Questo conferma il ruolo del tool non solo come strumento di supporto operativo, ma anche come risorsa con un **effetto educativo** per gli sviluppatori. La rilevanza di questo risultato è ulteriormente amplificata dal fatto che il campione era costituito in larga parte da studenti universitari, per i quali l'aspetto formativo rappresenta un valore aggiunto significativo. In linea con quanto osservato da Carver et al. [29], gli strumenti software possono infatti diventare veicoli di apprendimento, aiutando gli studenti a interiorizzare concetti e pratiche di qualità del codice. Analogamente, Nielsen [30] e Shneiderman et al. [31] sottolineano come il design di strumenti interattivi non si limiti a favorire l'usabilità, ma contribuisca anche a rafforzare conoscenze e buone pratiche attraverso il loro uso ripetuto.

Un ulteriore approfondimento riguarda le problematiche percepite come più gravi per ciascuno smell (Figura 5.16). Dai dati emerge che ogni smell viene associato a rischi specifici: ad esempio, lo smell GNC (Gradients Not Cleared) è stato collegato a possibili malfunzionamenti nell'addestramento, mentre IPA è stato percepito come fonte di errori logici e inconsistenze nei risultati.

Infine, la distribuzione delle risposte alla domanda su quale smell meritasse maggiore o minore attenzione (Figura 5.17) mostra come IPA e GNC siano considerati i più rilevanti, mentre CDE e TA sono percepiti come più contestuali e dipendenti dalla situazione, e PC come marginale. Alcuni partecipanti hanno esplicitamente segnalato questa dipendenza dal contesto: il Partecipante 3 ha affermato che *"CDE può essere pericoloso solo se i dataset sono di grandi dimensioni, altrimenti l'impatto è minimo"*, mentre il Partecipante 7 ha dichiarato che *"TA non è sempre critico, dipende dall'efficienza richiesta e dal tipo di modello"*.

Nel complesso, queste evidenze confermano un chiaro **contributo formativo del tool**: l'esperimento non si è limitato a valutare la capacità di detection, ma ha contribuito ad accrescere la consapevolezza degli sviluppatori riguardo alle implicazioni pratiche degli smell, consolidando l'utilità di *CodeSmile* anche come risorsa di apprendimento.

Domanda Survey: Post esperimento, presterai maggiore attenzione a che smell?

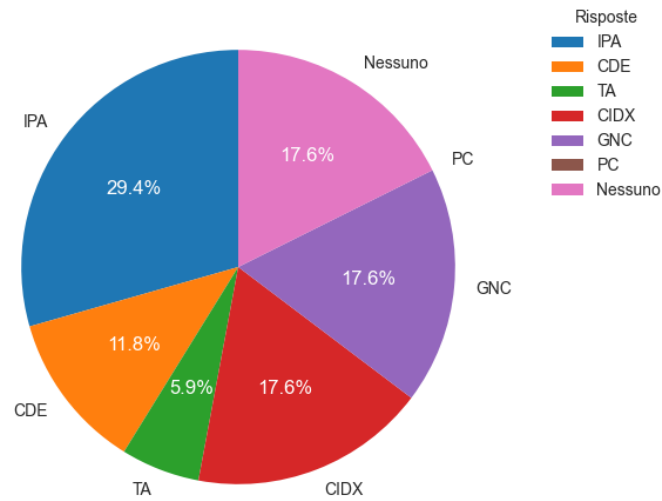


Figura 5.14: smell a cui i partecipanti dichiarano di prestare maggiore attenzione in futuro

Categorie di Effetti formativi e consapevolezza

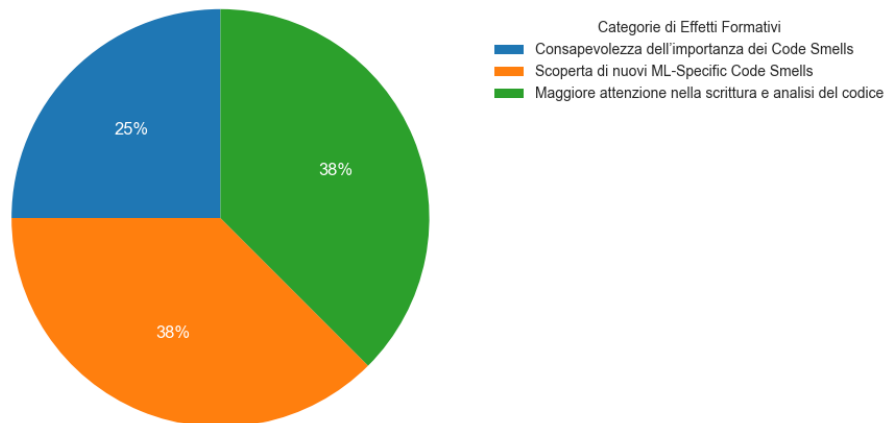


Figura 5.15: Categorie di consapevolezza ed effetti formativi percepiti dai partecipanti. Il grafico mostra le diverse dimensioni in cui i partecipanti hanno dichiarato di aver beneficiato dell'esperimento

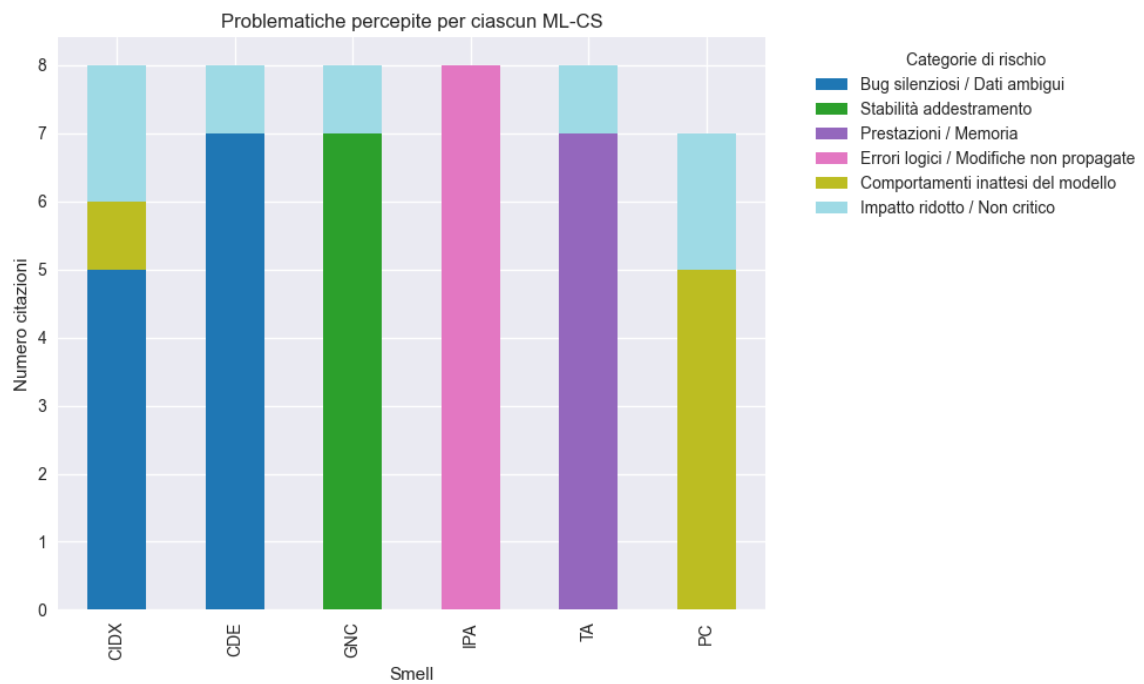


Figura 5.16: Problematiche percepite come più gravi per ciascun smell

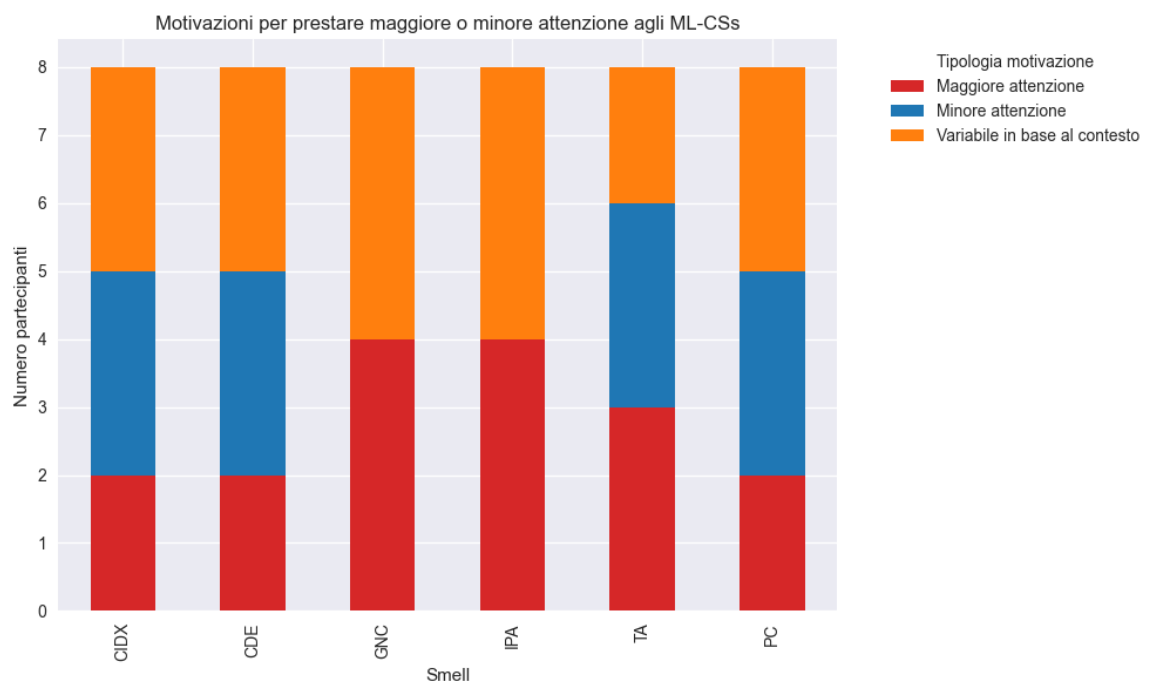


Figura 5.17: Distribuzione delle risposte su quale smell merita maggiore/minore attenzione

5.7 Metriche Soggettive dell'Esperimento

Accanto alle metriche oggettive, che hanno misurato tempi e accuratezza delle analisi, lo studio ha raccolto anche dati soggettivi relativi all'esperienza d'uso del plugin *CodeSmile*. Ai partecipanti è stato chiesto di esprimere giudizi su utilità, usabilità e propensione a consigliarne l'impiego, così da integrare le prestazioni quantitative con la percezione personale dello strumento.

5.7.1 Preferenze di Analisi e Raccomandazione

I risultati sono inequivocabili: la totalità dei partecipanti ha dichiarato di preferire l'uso del tool rispetto all'analisi manuale e di raccomandarne l'impiego ai colleghi. Questo dato evidenzia la percezione fortemente positiva dello strumento, che è stato visto come un supporto affidabile e utile nel lavoro di detection.

5.7.2 Utilità Percepita e Accordo con le Affermazioni

La valutazione dell'utilità percepita è riportata nel boxplot della Figura 5.18, che mostra una mediana pari a 4.5 su un massimo di 5 e una dispersione molto contenuta: i partecipanti hanno quindi giudicato l'esperienza con il plugin come altamente positiva.

Il multiple boxplot della Figura 5.19 mostra invece il livello di accordo con quattro affermazioni chiave:

1. La rilevazione tramite tool rende più semplice l'analisi e l'ispezione del codice.
2. Il tool ha rilevato smell che sarebbero potuti sfuggire manualmente.
3. I messaggi forniti dal tool erano chiari e comprensibili.
4. I smell individuati erano coerenti con il codice analizzato.

Tutte le mediane si collocano tra 4 e 5, indicando un'elevata soddisfazione generale. In particolare, l'affermazione che ha ricevuto il punteggio più alto è la prima, confermando che il contributo principale del plugin è stato quello di semplificare l'analisi e l'ispezione del codice. Le risposte più eterogenee si riscontrano invece nella

seconda e nella terza affermazione: alcuni partecipanti hanno apprezzato la capacità del tool di individuare smell altrimenti ignorati, mentre altri hanno evidenziato margini di miglioramento nella chiarezza dei messaggi e nella completezza delle segnalazioni.

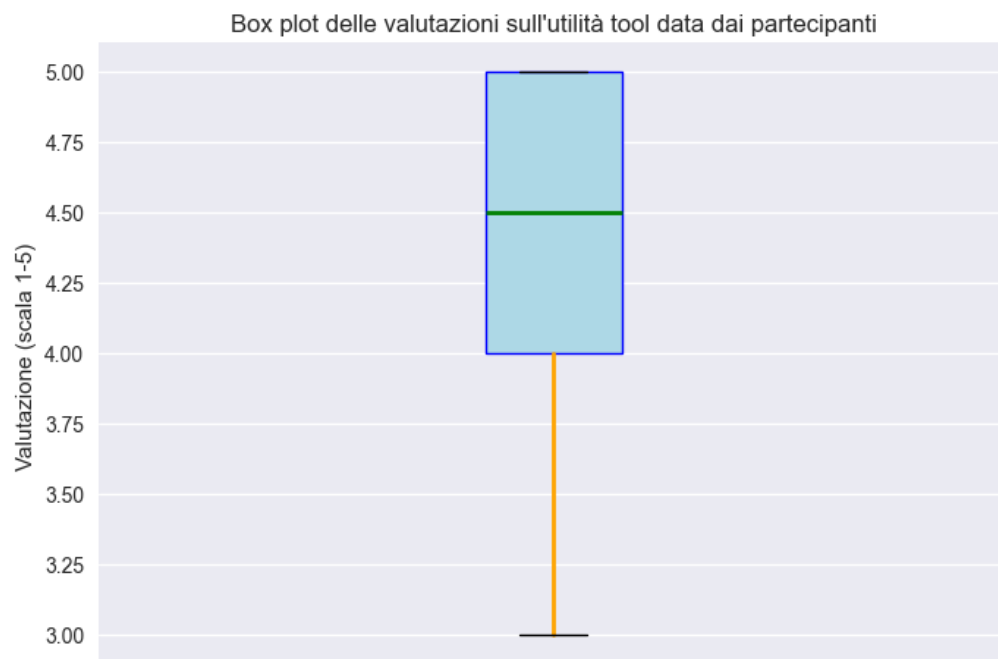


Figura 5.18: Valutazione dell'utilità percepita del tool

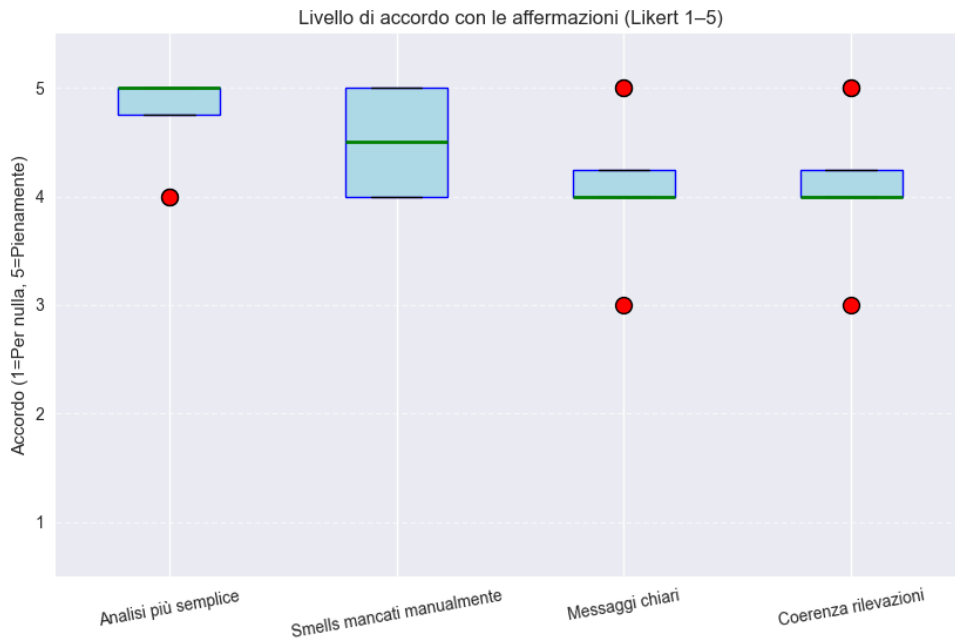


Figura 5.19: Livello di accordo con le affermazioni sull'esperienza d'uso del tool

5.7.3 Conoscenza Altre Problematiche e Motivazioni Preferenza Tool

Un'ultima serie di domande ha riguardato la conoscenza di altre problematiche di qualità del codice in ambito ML e le motivazioni dietro la preferenza per il tool.

Il pie chart di Figura 5.20 mostra come il 50% dei partecipanti non abbia dichiarato alcuna conoscenza di altre problematiche, mentre il restante 50% ha menzionato concetti quali manutenibilità del codice, *technical debt* [3], gestione della memoria GPU¹, e *data leakage*².

Le valutazioni sull'esperienza con il tool sono state raccolte in un bar chart (Figura 5.21), che evidenzia come le dimensioni più apprezzate siano state rapidità/efficienza e accuratezza/copertura, seguite da facilità d'uso e supporto all'apprendimento. Le criticità hanno invece ricevuto un punteggio marginale, segnalando

¹Problematiche legate all'uso intensivo delle GPU nei framework ML, che possono causare inefficienze o crash per esaurimento delle risorse

²Fenomeno in cui informazioni provenienti dal dataset di test contaminano il training, portando a stime di performance artificialmente elevate

un livello di soddisfazione complessivo molto alto, pur con alcuni spunti per futuri miglioramenti.

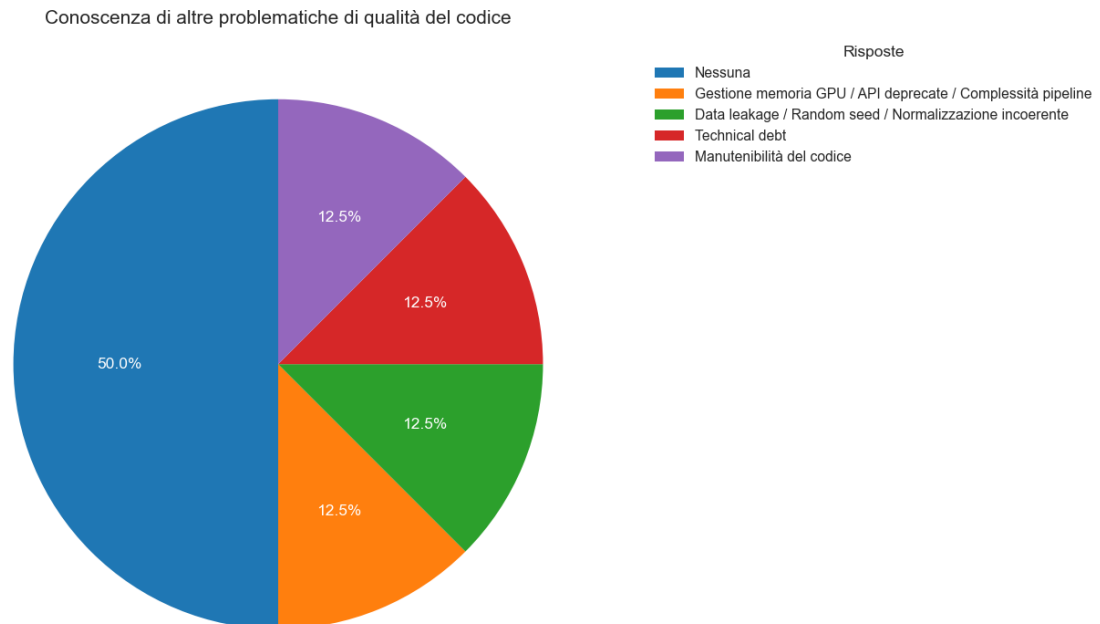


Figura 5.20: Conoscenza di altre problematiche di qualità in ambito ML dichiarata dai partecipanti

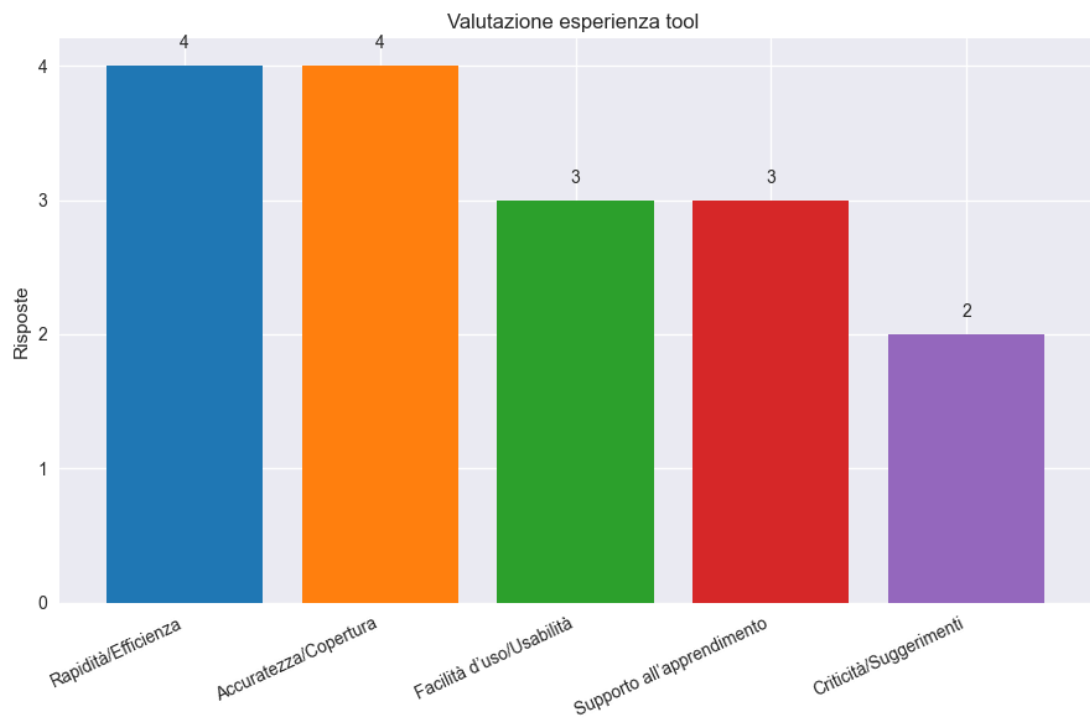


Figura 5.21: Valutazioni dei partecipanti sull'esperienza con il Tool

5.8 Discussione Complessiva

Alla luce dei risultati presentati nelle sezioni precedenti, è possibile fornire una risposta strutturata alle quattro *Research Questions* definite nel Capitolo 4.

🔗 **Answer to RQ₁.** In che misura l'utilizzo di CodeSmile incrementa l'efficacia nella rilevazione di ML-CSs rispetto a una rilevazione manuale?

Come discusso nella sezione 5.3 Metriche Oggettive dell'Esperimento, l'utilizzo di CodeSmile ha comportato un netto incremento dell'*accuracy* complessiva (94,3% vs 75,8%), con un F1-score medio passato da 81,9% a 96,3%. Questo miglioramento emerge in particolare sui True Positive (da circa 54% a quasi 95%) e sui True Negative (da 62,5% a 100%), confermando un guadagno significativo in termini di efficacia rispetto all'analisi manuale. Tali evidenze si allineano con i valori di precision e recall già riportati da Recupito et al. [5], a partire dai quali il presente lavoro ha esteso l'approccio con la realizzazione di un plugin integrato in ambiente IDE.

D'altro canto, il confronto ha anche messo in luce un fenomeno di *over-trust*, già documentato nella letteratura sull'affidamento eccessivo agli strumenti automatici [32]. Nello specifico, mentre nell'analisi manuale i partecipanti hanno talvolta saputo riconoscere i falsi positivi (37,5%), con il tool tale capacità critica si è azzerata (0%). Analogamente, nei casi di falsi negativi, i partecipanti hanno mostrato una minore propensione a contestare l'assenza di segnalazioni (21,9% vs 34,4% in modalità manuale). Nel complesso, CodeSmile incrementa l'efficacia della detection, ma l'elevata fiducia riposta nel tool introduce nuove criticità da considerare.

🔗 **Answer to RQ₂.** Quanto incide l'utilizzo di CodeSmile sulla riduzione del tempo necessario per la rilevazione di ML-CSs?

Come evidenziato dal boxplot dei tempi medi manuali (Figura 5.7), l'analisi richiedeva circa 4 minuti per file, con bassa variabilità tra i partecipanti. I file sottoposti all'ispezione avevano dimensioni molto simili, con una lunghezza compresa tra un minimo di 6 e un massimo di 28 righe di codice (in media circa 13–14 righe). Questo dato permette di interpretare i 4 minuti come un tempo tipico per l'ispezione di frammenti di codice relativamente brevi, non influenzato da variazioni significative nella dimensione dei file.

Al contrario, il boxplot dei tempi con il tool (Figura 5.8) mostra valori prossimi allo zero nella quasi totalità dei casi, evidenziando come l'analisi sia stata completata in maniera pressoché istantanea. Ciò determina un guadagno di efficienza che, seppur atteso, risulta sostanziale e rende l'attività di rilevazione decisamente più rapida rispetto all'analisi manuale.

È inoltre ragionevole ipotizzare che, nel caso di file di dimensioni molto maggiori (centinaia di righe), il divario a favore del tool sarebbe risultato ancora più marcato, amplificando ulteriormente il beneficio in termini di tempo.

🔗 **Answer to RQ₃.** Come viene percepita l'usabilità e l'utilità di CodeSmile dagli sviluppatori ML?

Il boxplot sull'utilità percepita (Figura 5.18) mostra una mediana di 4.5 su 5, con valori strettamente concentrati; il multiple boxplot delle affermazioni (Figura 5.19) conferma una soddisfazione elevata, in particolare per la semplificazione dell'analisi. Questa percezione positiva è in linea con i principi di usabilità e di valutazione dell'esperienza d'uso descritti nella letteratura HCI [22]. Sono emersi tuttavia margini di miglioramento legati alla chiarezza dei messaggi e alla gestione di casi complessi.

🔗 **Answer to RQ₄.** L'utilizzo di CodeSmile supporta l'apprendimento di concetti relativi ai code smell e accresce la percezione del rischio dei code smell nei sistemi ML-enabled?

I boxplot su pericolosità e frequenza (Figure 5.12, 5.13), insieme alla distribuzione delle priorità (Figura 5.17), indicano un'accresciuta consapevolezza dei rischi associati ai smell. IPA e GNC sono stati riconosciuti come smell più critici, mentre CDE e TA sono stati giudicati pericolosi solo in contesti specifici. Alcuni partecipanti hanno esplicitamente dichiarato che *"CDE può essere pericoloso solo se i dataset sono di grandi dimensioni"* (Partecipante 3) o che *"TA non è sempre critico, dipende dal modello"* (Partecipante 7).

Sul versante dell'*apprendimento di concetti relativi ai code smell*, i grafici e le risposte aperte hanno evidenziato che molti partecipanti hanno scoperto smell fino ad allora sconosciuti o hanno acquisito maggiore consapevolezza dei rischi: ad esempio, il Partecipante 2 ha affermato *"Ho imparato a prestare attenzione a smell che prima ignoravo"*, mentre il Partecipante 6 ha osservato *"Non conoscevo affatto IPA, ora so che può portare a errori logici seri"*. Questo effetto formativo è coerente con i fenomeni di apprendimento

riflessivo mediato dagli strumenti [33]. Ulteriori parallelismi possono essere tracciati con studi recenti di learning analytics, che mostrano come strumenti e dashboard possano facilitare la consapevolezza e la riflessione sui processi di apprendimento [34].

Sintesi dei risultati

- **Efficacia ed efficienza:** l'uso del plugin *CodeSmile* ha incrementato sensibilmente i True Positive e i True Negative, riducendo al contempo i tempi di analisi rispetto all'ispezione manuale.
- **Valutazioni soggettive:** i partecipanti hanno espresso una percezione fortemente positiva in termini di utilità, usabilità e propensione a consigliare lo strumento.
- **Over-trust:** l'esperimento ha evidenziato casi di fiducia eccessiva verso le segnalazioni del tool, indicando la necessità di maggiore trasparenza e chiarezza nei messaggi per stimolare un uso critico.
- **Effetto formativo:** *CodeSmile* si è rivelato non solo un rilevatore automatico di ML-CSs, ma anche un alleato didattico, in grado di accrescere la consapevolezza degli sviluppatori rispetto alle problematiche di qualità del codice in ambito ML.

Conclusioni e Sviluppi Futuri

6.1 Conclusioni

Il lavoro presentato in questa tesi ha affrontato una problematica ancora poco esplorata ma di crescente rilevanza: la rilevazione automatica dei *Machine Learning-specific Code smell* (ML-CSs). Attraverso lo sviluppo di CODESMILE, un plugin per JetBrains PyCharm, si è reso possibile portare all'interno dell'ambiente di sviluppo funzionalità di detection *on-demand* e *real-time*, superando uno dei limiti principali degli strumenti esistenti, quasi esclusivamente offline [5].

I risultati dello *user study controllato* hanno confermato l'utilità e l'efficacia percepita dello strumento. In particolare:

- l'analisi con il plugin ha incrementato la percentuale di *True Positive* e *True Negative*, riducendo i tempi di detection a pochi secondi rispetto ai circa 4 minuti richiesti per l'analisi manuale;
- la fiducia verso lo strumento è stata mediamente alta, con una tendenza tuttavia a fenomeni di *over-trust*, in cui i partecipanti si sono spesso allineati alle predizioni del tool anche in presenza di falsi positivi o negativi;

- l'esperimento ha evidenziato un chiaro *accrescimento della consapevolezza e conoscenza sull'argomento ML-CSs*: diversi partecipanti hanno dichiarato di aver acquisito maggiore consapevolezza sull'importanza degli ML-CSs e sulla necessità di prevenire cattive pratiche durante lo sviluppo.

Nel complesso, i risultati ottenuti rispondono positivamente alle *Research Questions*: CODESMILE si è dimostrato uno strumento in grado di aumentare l'accuratezza, ridurre i tempi di analisi, supportare l'apprendimento e fornire un contributo concreto all'integrazione di pratiche di *continuous quality assurance* nello sviluppo di sistemi ML-enabled.

6.2 Sviluppi Futuri

Il lavoro svolto con CODESMILE rappresenta un primo passo verso l'integrazione di strumenti di rilevamento dei ML-CSs direttamente all'interno dei flussi di sviluppo. Tuttavia, sono diverse le direzioni in cui questo progetto potrà evolversi nei prossimi anni, sia dal punto di vista tecnico che metodologico.

Un primo obiettivo sarà la **distribuzione del plugin sul JetBrains Marketplace**, rendendolo facilmente accessibile alla comunità di sviluppatori. In parallelo, si prevede la predisposizione di un **server remoto per la raccolta di log e metriche di utilizzo**, così da poter monitorare casi reali d'impiego del tool e raccogliere evidenze empiriche preziose per la sua evoluzione. In questo modo, il plugin potrà crescere non solo come progetto di ricerca ma anche come strumento pratico di supporto allo sviluppo industriale.

Un secondo ambito di estensione riguarda l'**integrazione di modelli basati su Large Language Models (LLM)**. L'attuale versione di CODESMILE si fonda su un approccio rule-based su AST, che garantisce efficienza e interpretabilità. Tuttavia, i recenti progressi dei LLM per l'analisi del codice aprono prospettive interessanti per la rilevazione di smell più complessi e contestuali, difficilmente catturabili da semplici regole statiche. Se finora questa direzione non è stata perseguita a causa delle limitazioni hardware e della complessità computazionale, in futuro sarà pos-

sibile valutare approcci ibridi regole+AI, capaci di coniugare leggerezza e potenza predittiva.

Un terzo sviluppo naturale riguarda l'**estensione della copertura del catalogo ML-CSs**. Ad oggi il plugin è in grado di rilevare 12 dei 22 smell identificati nello studio di Zhang et al. [2]. L'intento è ampliare progressivamente la copertura, includendo le tipologie mancanti e affinando i rilevatori già esistenti, così da garantire un'analisi sempre più completa e accurata. Va inoltre sottolineato che il catalogo dei ML-CSs è in continua evoluzione: nuovi smell potrebbero emergere dalla ricerca nei prossimi anni, ed è obiettivo del progetto mantenere il plugin aggiornato per riflettere in modo fedele lo stato dell'arte.

Un ulteriore passo sarà l'**integrazione con i workflow dei sistemi di version control**, come GitHub o GitLab. L'idea è quella di permettere al plugin di analizzare automaticamente il codice in fase di commit o pull request, rilevando eventuali smell introdotti. Questa funzionalità consentirebbe di trasformare CodeSmile in uno strumento non solo di supporto allo sviluppo locale, ma anche di garanzia di qualità continua nei processi di *code review* e integrazione continua, avvicinandolo ulteriormente alle pratiche di DevOps.

Infine, un passaggio cruciale sarà la **validazione in contesti industriali**. Lo *user study* condotto in questa tesi ha avuto natura esplorativa e accademica, basandosi principalmente su un campione di studenti e giovani sviluppatori. Per consolidare la maturità dello strumento, sarà importante testarlo in scenari produttivi reali, confrontandosi con esigenze, carichi di lavoro e criticità tipiche di progetti industriali di medio-grandi dimensioni. Solo in questo modo sarà possibile raccogliere feedback avanzati e validare l'efficacia del tool su un pubblico di sviluppatori esperti.

In definitiva, il plugin CODESMILE rappresenta un primo passo verso strumenti di quality assurance per il Machine Learning più maturi e integrati nei flussi reali di sviluppo. La sua evoluzione nei prossimi anni potrà rafforzarne il ruolo come supporto concreto agli sviluppatori, sia in ambito accademico che industriale.

Bibliografia

- [1] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Boston, MA, USA: Addison-Wesley, 1999. (Citato alle pagine 1 e 5)
- [2] H. Zhang, L. Cruz, and A. Van Deursen, “Code smells for machine learning applications,” in *Proceedings of the 1st international conference on AI engineering: software engineering for AI*, 2022, pp. 217–228. (Citato alle pagine 1, 5, 6, 9, 11, 12, 14, 17 e 70)
- [3] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden technical debt in machine learning systems,” *Advances in neural information processing systems*, vol. 28, 2015. (Citato alle pagine 1, 7 e 63)
- [4] H. Zhang, “Automated detection of code smells for machine learning applications,” Ph.D. dissertation, Department of Software Technology Faculty EEMCS, Delft University of Technology, 2022. (Citato alle pagine 2, 12 e 15)
- [5] G. Recupito, G. Giordano, F. Ferrucci, D. Di Nucci, and F. Palomba, “When code smells meet ml: on the lifecycle of ml-specific code smells in ml-enabled systems,” *Empirical Software Engineering*, vol. 30, no. 5, pp. 1–49, 2025. (Citato alle pagine 2, 6, 8, 10, 11, 12, 14, 15, 17, 18, 36, 55, 56, 65 e 68)

-
- [6] H. Jebnoun, H. Ben Braiek, M. M. Rahman, and F. Khomh, "The scent of deep learning code: An empirical study," in *Proceedings of the 17th International Conference on Mining Software Repositories*, 2020, pp. 420–430. (Citato alle pagine 6, 9 e 14)
- [7] N. Cardozo, I. Dusparic, and C. Cabrera, "Prevalence of code smells in reinforcement learning projects," *arXiv preprint arXiv:2303.10236*, 2023. (Citato alle pagine 6, 9 e 14)
- [8] P. Avgeriou, P. Kruchten, I. Ozkaya, and C. Seaman, "Managing technical debt in software engineering (dagstuhl seminar 16162)," *Dagstuhl reports*, vol. 6, no. 4, pp. 110–138, 2016. (Citato a pagina 6)
- [9] P.-O. Côté, A. Nikanjam, R. Bouchoucha, I. Basta, M. Abidi, and F. Khomh, "Quality issues in machine learning software systems," *Empirical Software Engineering*, vol. 29, no. 6, p. 149, 2024. (Citato alle pagine 6, 9, 10, 14, 15, 17 e 18)
- [10] J. Bogner, R. Verdecchia, and I. Gerostathopoulos, "Characterizing technical debt and antipatterns in ai-based systems: A systematic mapping study," in *2021 IEEE/ACM International Conference on Technical Debt (TechDebt)*. IEEE, 2021, pp. 64–73. (Citato alle pagine 7 e 15)
- [11] R. Sandouka and H. Aljamaan, "Python code smells detection using conventional machine learning models," *PeerJ Computer Science*, vol. 9, p. e1370, 2023. (Citato a pagina 12)
- [12] A. Ho, A. M. T. Bui, P. T. Nguyen, A. D. Salle, and B. Le, "Ensesmells: Deep ensemble and programming language models for automated code smells detection," 2025. [Online]. Available: <https://arxiv.org/abs/2502.05012> (Citato alle pagine 12 e 15)
- [13] P. Hamfelt, R. Britto, L. Rocha, and C. Almendra, "Automatic identification of machine learning-specific code smells," 2025. [Online]. Available: <https://arxiv.org/abs/2508.02541> (Citato alle pagine 12 e 13)

-
- [14] K. Shivashankar and A. Martini, "MIscent a tool for anti-pattern detection in ml projects," 2025. [Online]. Available: <https://arxiv.org/abs/2502.18466> (Citato alle pagine 12 e 15)
- [15] B. Bruegge and A. H. Dutoit, *Object oriented software engineering: Using UML patterns and Java*. Prentice Hall, 2010. (Citato alle pagine 18, 21, 23 e 26)
- [16] I. Sommerville, *Software Engineering*, 10th ed., M. Horton, M. Hirsch, M. Goldstein, C. Bell, M. Borthakur, and J. Holcomb, Eds. Pearson, 2016. [Online]. Available: <https://www.pearson.com/us/higher-education/program/PGM35255.html> (Citato alle pagine 18, 20, 21 e 26)
- [17] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design patterns: elements of reusable object-oriented software*. Pearson Deutschland GmbH, 1995. (Citato alle pagine 18, 21 e 27)
- [18] R. S. Pressman, *Software engineering: a practitioner's approach*. Palgrave macmillan, 2005. (Citato alle pagine 21, 23, 25 e 27)
- [19] M. Fowler, *Refactoring: improving the design of existing code*. Addison-Wesley Professional, 2018. (Citato a pagina 24)
- [20] E. Freeman, E. Robson, B. Bates, and K. Sierra, *Head First Design Patterns: A Brain-Friendly Guide*. "O'Reilly Media, Inc.", 2004. (Citato a pagina 27)
- [21] V. Stodden, M. McNutt, D. H. Bailey, E. Deelman, Y. Gil, B. Hanson, M. A. Heroux, J. P. Ioannidis, and M. Taufer, "Enhancing reproducibility for computational methods," *Science*, vol. 354, no. 6317, pp. 1240–1241, 2016. (Citato a pagina 27)
- [22] J. Lazar, J. H. Feng, and H. Hochheiser, *Research methods in human-computer interaction*. Morgan Kaufmann, 2017. (Citato alle pagine 29, 30, 34, 55 e 66)
- [23] H. Müller, A. Sedley, and E. Ferrall-Nunge, "Designing unbiased surveys for hci research," in *CHI'14 Extended Abstracts on Human Factors in Computing Systems*, 2014, pp. 1027–1028. (Citato alle pagine 29, 31 e 34)

- [24] A. Følstad, "Users' design feedback in usability evaluation: a literature review," *Human-centric Computing and Information Sciences*, vol. 7, no. 1, p. 19, 2017. (Citato alle pagine 29 e 31)
- [25] P. Kashfi, K. Kuusinen, and R. Feldt, "Stakeholder involvement: A success factor for achieving better ux integration," 2016. [Online]. Available: <https://arxiv.org/abs/1610.04774> (Citato a pagina 29)
- [26] M. Nasir, "Crowdsourcing: A framework for usability evaluation," *arXiv preprint arXiv:2408.06955*, 2024. (Citato a pagina 29)
- [27] A. Joshi, S. Kale, S. Chandel, and D. K. Pal, "Likert scale: Explored and explained," *British journal of applied science & technology*, vol. 7, no. 4, p. 396, 2015. (Citato a pagina 34)
- [28] R. Parasuraman and V. Riley, "Humans and automation: Use, misuse, disuse, abuse," *Human factors*, vol. 39, no. 2, pp. 230–253, 1997. (Citato a pagina 36)
- [29] J. Carver, L. Jaccheri, S. Morasca, and F. Shull, "Issues in using students in empirical studies in software engineering education," in *Proceedings. 5th international workshop on enterprise networking and computing in healthcare industry (IEEE Cat. No. 03EX717)*. IEEE, 2004, pp. 239–249. (Citato a pagina 58)
- [30] J. Nielsen, *Usability Engineering*. San Francisco, CA: Morgan Kaufmann, 1993. (Citato a pagina 58)
- [31] B. Shneiderman, C. Plaisant, M. Cohen, S. Jacobs, N. Elmqvist, and N. Diakopoulos, *Designing the User Interface: Strategies for Effective Human-Computer Interaction*, 6th ed. Boston, MA: Pearson, 2017. (Citato a pagina 58)
- [32] M. T. Dzindolet, S. A. Peterson, R. A. Pomranky, L. G. Pierce, and H. P. Beck, "The role of trust in automation reliance," *International journal of human-computer studies*, vol. 58, no. 6, pp. 697–718, 2003. (Citato a pagina 65)
- [33] D. A. Schön, *The reflective practitioner: How professionals think in action*. Routledge, 2017. (Citato a pagina 67)

- [34] K. Zhang, R. Yılmaz, A. B. Ustun, and F. G. K. Yılmaz, "Learning analytics in formative assessment: A systematic literature review," *Journal of Measurement and Evaluation in Education and Psychology*, vol. 14, no. Özel Sayı, pp. 359–381, 2023. (Citato a pagina 67)

Ringraziamenti

Ringraziamenti qui...